

TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU THỰC HÀNH

Học máy ứng dụng

Bài 1: Hồi quy tuyến tính

Biên soạn ThS. Nguyễn Thái Anh

Năm học 2026 – 2027

Mục lục

1	Mục tiêu bài thực hành	3
2	Chuẩn bị môi trường làm việc	4
2.1	Bốn thư viện sẽ dùng	4
2.2	Bước 1: Kiểm tra máy đã có Python chưa	4
2.3	Bước 2: Tạo thư mục cho môn học	4
2.4	Bước 3: Tạo môi trường ảo	5
2.5	Bước 4: Cài bốn thư viện	5
2.6	Bước 5: Kiểm tra đã cài được chưa	6
2.7	Bước 6: Cấu trúc thư mục bài thực hành	6
2.8	Bước 7: Chạy thử tệp đầu tiên	7
3	Làm quen bộ dữ liệu	7
3.1	Bài toán đặt ra	7
3.2	Bốn cột trong tệp dữ liệu	8
3.3	Đọc dữ liệu bằng pandas	8
3.4	Nhìn dữ liệu trước khi tính toán	10
4	Hồi quy tuyến tính là gì	12
4.1	Ý tưởng chỉ gói trong một đường thẳng	12
4.2	Ba đường thẳng, đường nào đúng	13
4.3	Đo sai số của một căn nhà	13
4.4	Từ sai số của từng căn tới một con số duy nhất	14
4.5	Tính thử bằng tay trên năm căn nhà	15
4.6	Bài toán thật sự cần giải	17
5	Tìm đường thẳng tốt nhất	18
5.1	Cách 1: dùng công thức có sẵn	18
5.2	Cách 2: để scikit-learn làm	20
6	Đánh giá mô hình	23
6.1	Vì sao không được chấm điểm trên chính dữ liệu đã học	23
6.2	Chia tập học và tập kiểm tra	24
6.3	Bốn con số để chấm điểm	25
6.4	Chấm điểm mô hình của chúng ta	27
6.5	Nhìn sai số bằng hình	29
7	Máy tự dò lời giải bằng gradient descent	31
7.1	Vì sao vẫn cần cách này	31
7.2	Ý tưởng người mù xuống núi	32
7.3	Ba dòng cốt lõi	32
7.4	Một bước chuẩn bị không được bỏ	33
7.5	Chạy thử và đối chiếu với đáp án	33
7.6	Tốc độ học, tham số phải tự chọn	37

8	Dùng nhiều hơn một biến	39
8.1	Từ một đường thẳng sang một mặt phẳng	39
8.2	Trong mã chỉ đòi đúng một chỗ	39
8.3	Đọc hệ số của từng biến	41
8.4	Một giới hạn phải nhớ	42
9	Bài tập	43
10	Phụ lục A, bảng tra nhanh	45
10.1	Các hàm đã dùng trong bài	45
10.2	Các ký hiệu toán dùng trong bài	46
10.3	Các con số chính của bài	47
11	Phụ lục B, lỗi thường gặp	48

1. Mục tiêu bài thực hành

Trong bài này sinh viên sẽ dựng một mô hình dự đoán giá căn hộ chỉ từ diện tích, làm bằng tay trước rồi mới gọi thư viện. Bộ dữ liệu đi kèm gồm 60 căn hộ, mỗi căn có bốn con số là diện tích, số phòng, tuổi nhà và giá bán. Kết thúc ba tiết thực hành, sinh viên sẽ có một mô hình đoán lệch trung bình 0.2578 tỷ đồng trên những căn mà nó chưa từng nhìn thấy. Con số đó không phải là đích đến, cái sinh viên cần mang về là hiểu vì sao máy tìm ra được đường thẳng ấy.

Sau buổi thực hành này, sinh viên phải tự làm được bảy việc dưới đây.

1. Giải thích được hồi quy tuyến tính (*linear regression*) là gì, và một đường thẳng vẽ xuyên qua đám điểm dữ liệu thì có ý nghĩa gì.
2. Đọc được một tệp dữ liệu dạng CSV bằng thư viện pandas, xem được bảng có bao nhiêu dòng bao nhiêu cột, và kiểm tra được bảng có ô trống hay không.
3. Tính được sai số của một dự đoán, rồi gộp toàn bộ sai số của 60 căn thành một con số duy nhất gọi là MSE.
4. Dùng được hai cách khác nhau để tìm hệ số góc w và hệ số chặn b của đường thẳng, một là công thức tính tay, hai là thư viện scikit-learn, và tự kiểm chứng rằng hai cách cho ra cùng một kết quả $w = 0.078367$ và $b = 0.401752$.
5. Đánh giá được mô hình cho hàng hoàng, nghĩa là chia dữ liệu thành tập học và tập kiểm tra rồi đọc ba chỉ số MAE, RMSE và R^2 , ba chỉ số này sẽ được giải thích từng cái một ở Mục 6.
6. Nhận ra được ý tưởng của gradient descent, tức là cách máy tự dò dần tới lời giải khi trong tay không có sẵn công thức.
7. Viết được một mô hình dùng nhiều hơn một biến, cụ thể là dùng cả ba cột diện tích, số phòng và tuổi nhà thay vì chỉ một cột diện tích.

Cách học bài này

Mọi con số và mọi hình trong tài liệu này đều là kết quả chạy thật trên bộ dữ liệu kèm theo, không có con số nào chép từ sách và cũng không có con số nào được làm tròn cho đẹp. Vì vậy sinh viên hãy gõ lại từng đoạn mã vào máy mình rồi chạy, sau đó so kết quả trên màn hình của mình với kết quả in trong tài liệu. Hai bên phải khớp nhau tới từng chữ số. Nếu lệch thì đừng vội gọi thầy. Hãy đọc kỹ thông báo lỗi trước, vì dòng cuối cùng của thông báo lỗi gần như luôn nói thẳng ra tệp nào và dòng nào đang sai.

Bảng 1: Bài này cần gì trước, và cái gì thì không cần.

Nội dung	Mức độ cần có
Chạy một tệp Python	Biết mở cửa sổ lệnh rồi gõ <code>python ten_tep.py</code>
Biến và phép gán	Hiểu một dòng kiểu <code>x = 5</code> làm gì
Vòng lặp <code>for</code>	Đọc hiểu một vòng lặp chạy qua một danh sách
Mảng và danh sách	Biết phần tử đầu tiên mang chỉ số 0
Kiến thức học máy	Không cần, bài này bắt đầu từ con số không

2. Chuẩn bị môi trường làm việc

Phần này không dạy học máy, nó chỉ lo cho cái máy của sinh viên chạy được. Nhưng sinh viên đừng bỏ qua, vì phần lớn thời gian chết trong buổi thực hành đầu tiên là do máy chưa cài đúng chứ không phải do bài khó. Hãy làm lần lượt bảy bước bên dưới, làm xong bước nào thì kiểm tra bước đó rồi mới sang bước sau. Toàn bộ phần này sinh viên chỉ phải làm một lần duy nhất cho cả môn học.

2.1. Bốn thư viện sẽ dùng

Chúng ta sẽ cài bốn thư viện (*library*), mỗi thư viện lo một việc riêng. Sinh viên hãy đọc qua Bảng 2 một lượt cho biết tên, chưa cần hiểu sâu.

Bảng 2: Bốn thư viện dùng trong bài và việc của từng thư viện.

Thư viện	Làm việc gì cho ta
numpy	Tính toán trên cả một dãy số cùng lúc, thay vì viết vòng lặp qua từng số
pandas	Đọc tệp CSV và giữ dữ liệu dưới dạng bảng có tên cột, giống một trang Excel
matplotlib	Vẽ hình, mọi hình trong tài liệu này đều do nó vẽ ra
scikit-learn	Chứa sẵn các mô hình học máy, gọi ba dòng là có mô hình

2.2. Bước 1: Kiểm tra máy đã có Python chưa

Sinh viên mở cửa sổ lệnh (*command prompt*) bằng cách bấm phím `Windows`, gõ chữ `cmd`, rồi bấm `Enter`. Một cửa sổ nền đen hiện ra. Sinh viên gõ dòng sau rồi bấm `Enter`.

```
python --version
```

Nếu máy in ra một dòng kiểu `Python 3.11.9` thì máy đã có Python và sinh viên sang Bước 2 được. Bài này cần Python phiên bản 3.11 trở lên, vì bốn thư viện chúng ta sắp cài đã bỏ hỗ trợ những bản cũ hơn. Nếu máy báo không tìm thấy lệnh, nghĩa là máy sinh viên chưa có Python. Khi đó sinh viên vào trang python.org, mục `Downloads`, tải bản mới nhất cho Windows rồi cài đặt.

Đừng bỏ sót ô này khi cài Python

Ở màn hình đầu tiên của bộ cài Python có một ô nhỏ tên là `Add Python to PATH` nằm dưới đáy cửa sổ. Sinh viên bắt buộc phải tích vào ô đó trước khi bấm nút cài. Nếu không tích, Python vẫn được cài vào máy nhưng cửa sổ lệnh sẽ không biết đường tìm tới nó, và sinh viên sẽ gặp lại đúng thông báo không tìm thấy lệnh ở trên. Lỡ quên rồi thì cách nhanh nhất là chạy lại bộ cài, chọn `Modify` và tích lại ô đó.

2.3. Bước 2: Tạo thư mục cho môn học

Sinh viên tạo một thư mục riêng cho cả môn học, ví dụ `C:\MayHoc`. Sinh viên nên chọn một đường dẫn ngắn và không có dấu tiếng Việt, cũng không có dấu cách. Lý do là một số công cụ dòng lệnh xử lý đường dẫn có dấu rất tệ, và lỗi đó rất khó đoán ra khi mới học. Có thư mục riêng còn một cái lợi nữa, đó là mọi thứ của môn này nằm gọn một chỗ nên khi cần xóa đi cài lại sinh viên chỉ xóa đúng một thư mục.

Sinh viên quay lại cửa sổ lệnh và gõ hai dòng sau, mỗi dòng bấm `Enter` một lần.

```
mkdir C:\MayHoc  
cd C:\MayHoc
```

Sau dòng thứ hai, dấu nhắc bên trái đổi thành `C:\MayHoc>`. Dấu nhắc đó cho sinh viên biết cửa sổ lệnh đang đứng ở thư mục nào, và đây là thông tin phải liếc qua thường xuyên.

2.4. Bước 3: Tạo môi trường ảo

Môi trường ảo (*virtual environment*) là một ngăn tủ riêng để chứa thư viện của một môn học. Sinh viên hãy hình dung máy sinh viên là một cái tủ lớn dùng chung cho mọi môn. Nếu môn này cần scikit-learn bản mới còn một đồ án khác cần bản cũ, hai bên đè lên nhau và cả hai cùng hỏng. Môi trường ảo cắt cho môn này một ngăn riêng, cái gì trong ngăn đó thì chỉ nằm trong ngăn đó, nên hỏng cũng không lan ra ngoài.

Sinh viên gõ lệnh sau để tạo ngăn tủ tên là `hocmay`.

```
python -m venv hocmay
```

Lệnh này chạy vài giây và không in ra gì cả, đó là bình thường. Nó tạo trong `C:\MayHoc` một thư mục con tên `hocmay`, bên trong là một bản Python riêng của sinh viên. Tạo xong thì ngăn tủ vẫn đang đóng, sinh viên phải mở nó ra bằng lệnh sau.

```
hocmay\Scripts\activate
```

Dấu hiệu để biết đã bật thành công là đầu dòng lệnh mọc thêm chữ (`hocmay`) trong ngoặc đơn, thành (`hocmay`) `C:\MayHoc>`. Chừng nào còn thấy chữ đó thì sinh viên đang làm việc trong ngăn tủ riêng. Nếu không thấy chữ đó, mọi thứ sinh viên cài sẽ chui ra ngoài tủ chung.

Mỗi lần mở cửa sổ lệnh mới đều phải bật lại

Môi trường ảo chỉ sống trong đúng cửa sổ lệnh mà sinh viên đã bật nó. Đóng cửa sổ đi rồi mở lại, hoặc mở thêm một cửa sổ thứ hai, thì chữ (`hocmay`) không tự có. Cho nên mỗi buổi ngồi vào máy, việc đầu tiên của sinh viên là `cd` về `C:\MayHoc` rồi chạy lại lệnh bật ở trên. Bước tạo bằng `python -m venv` thì ngược lại, chỉ làm đúng một lần trong đời, không phải làm lại.

2.5. Bước 4: Cài bốn thư viện

Khi đã thấy chữ (`hocmay`) ở đầu dòng, sinh viên gõ một lệnh duy nhất sau đây để cài cả bốn thư viện một lượt.

```
pip install numpy pandas matplotlib scikit-learn
```

Đây là lệnh chạy lâu nhất trong cả mục này, vì máy phải tải về vài chục megabyte từ mạng. Màn hình sẽ trôi liên tục các dòng chữ báo đang tải và đang cài, sinh viên cứ để yên và đừng đóng cửa sổ. Cài xong, dòng gần cuối bắt đầu bằng chữ `Successfully installed` và liệt kê tên các gói (*package*) vừa cài. Sinh viên để ý sẽ thấy số gói được cài nhiều hơn bốn, đó là vì bốn thư viện ta xin cài còn kéo theo vài thư viện phụ mà chúng cần.

2.6. Bước 5: Kiểm tra đã cài được chưa

Cài xong chưa chắc đã dùng được, nên ta viết một tệp nhỏ để hỏi thẳng từng thư viện. Sinh viên mở một trình soạn thảo bất kỳ, ví dụ Notepad hay Visual Studio Code, gõ đúng năm dòng dưới đây rồi lưu vào C:\MayHoc với tên `kiem_tra.py`.

```
import numpy, pandas, matplotlib, sklearn
print("numpy      ", numpy.__version__)
print("pandas     ", pandas.__version__)
print("matplotlib  ", matplotlib.__version__)
print("scikit-learn", sklearn.__version__)
```

Sinh viên để ý tên dùng khi gọi scikit-learn trong mã là `sklearn`, khác với tên dùng khi cài. Đây là điểm hay làm sinh viên bối rối, vì tên lúc cài là `scikit-learn` nhưng tên lúc nhập vào mã lại là `sklearn`. Lưu tệp xong, sinh viên quay ra cửa sổ lệnh và chạy nó.

```
python kiem_tra.py
```

Màn hình phải in ra bốn dòng, mỗi dòng là tên một thư viện kèm số phiên bản của nó. In đủ bốn dòng nghĩa là môi trường đã ổn. Nếu máy báo `ModuleNotFoundError` kèm tên một thư viện, thì gần như chắc chắn sinh viên đã quên bật môi trường ảo ở Bước 3 trước khi cài ở Bước 4. Cách sửa là bật lại môi trường ảo rồi chạy lại lệnh cài.

2.7. Bước 6: Cấu trúc thư mục bài thực hành

Sinh viên giải nén gói bài thực hành mà thầy phát vào trong C:\MayHoc. Sau khi giải nén, cây thư mục của sinh viên trông như sau.

```
C:\MayHoc\
  hocmay\
    bai01_hoi_quy\
      data\
        gia_nha.csv      60 can ho, 4 cot
      code\
        b1_doc_du_lieu.py
        b2_do_sai_so.py
        b3_cong_thuc.py
        b4_sklearn.py
        b5_danh_gia.py
        b6_gradient_descent.py
        b7_nhieu_bien.py
      figures\           cac hinh ve sinh ra
      outputs\           ket qua in ra man hinh cua tung buoc
      scripts\           tien ich chay lai toan bo
```

Bảy tệp trong `code` tương ứng với bảy bước của bài, đi từ `b1` tới `b7` theo đúng thứ tự các mục phía sau. Thư mục `outputs` chứa kết quả thật của lần chạy gần nhất, và tài liệu này chép nguyên nội dung các tệp đó vào. Nhờ vậy cái sinh viên đọc trong tài liệu đúng bằng cái máy in ra, không qua tay người gõ lại.

Lỗi hay gặp nhất cả buổi

Mọi lệnh python trong tài liệu này đều chạy từ thư mục gốc của bài thực hành, tức là từ `bai01_hoi_quy`. Sinh viên không được `cd` vào trong code rồi chạy tệp ở đó. Lý do rất cụ thể, trong mã nguồn đường dẫn tới dữ liệu được viết là `data/gia_nha.csv`, và Python hiểu đường dẫn đó so với thư mục sinh viên đang đứng chứ không phải so với chỗ đặt tệp mã. Đứng sai chỗ thì Python báo `FileNotFoundError` và nói không tìm thấy `data/gia_nha.csv`, dù tệp đó vẫn nằm yên trong máy sinh viên. Gặp thông báo đó, việc đầu tiên sinh viên làm là nhìn dấu nhắc xem mình đang đứng ở thư mục nào.

2.8. Bước 7: Chạy thử tệp đầu tiên

Sinh viên `cd` vào thư mục gốc của bài rồi chạy tệp đầu tiên.

```
cd C:\MayHoc\bai01_hoi_quy
python code/b1_doc_du_lieu.py
```

Nếu màn hình hiện ra một bảng số có các cột `dien_tich`, `so_phong`, `tuoi_nha` và `gia` thì môi trường đã sẵn sàng và sinh viên có thể đóng mục này lại. Ở đây sinh viên chưa cần hiểu bảng đó nói gì, cũng chưa cần đọc từng con số. Mục 3 sẽ mở xẻ kết quả này từng dòng một.

Vì sao chữ in ra màn hình trong các tệp mã đều không dấu

Sinh viên mở tệp mã ra sẽ thấy một chuyện lạ, phần chú thích thì tiếng Việt có dấu đầy đủ, còn phần chữ in ra màn hình thì viết không dấu kiểu `Kich thuoc bang`. Đây là cố ý, không phải người viết lười bỏ dấu. Cửa sổ lệnh của Windows theo mặc định dùng một bảng mã cũ, nên chữ tiếng Việt có dấu in ra thường thành ký tự lạ, có máy còn dừng chương trình giữa chừng và báo lỗi mã hóa. Phần chú thích thì khác, nó chỉ nằm trong tệp cho người đọc chứ không bao giờ đi ra màn hình, mà trình soạn thảo thì đọc được tiếng Việt, nên chú thích giữ nguyên dấu cho sinh viên dễ hiểu. Khi tự viết mã sau này, sinh viên cứ theo đúng quy ước đó cho tới khi học cách đổi bảng mã của cửa sổ lệnh.

3. Làm quen bộ dữ liệu

Trong học máy, việc đầu tiên không phải là viết mô hình. Việc đầu tiên là mở dữ liệu ra xem nó có gì. Mục này làm đúng một chuyện đó, và chỉ khi nào chúng ta hiểu rõ 60 dòng dữ liệu trong tay thì mới sang mục sau.

3.1. Bài toán đặt ra

Hãy hình dung một tình huống rất đời thường. Một người quen đang tìm mua căn hộ và nhắm được một căn rộng 80 m^2 . Người bán ra giá 7.5 tỷ đồng. Người quen đó hỏi sinh viên: mức giá này có hợp lý không?

Nếu trong tay có giá của 60 căn hộ khác trong cùng khu vực, sinh viên hoàn toàn trả lời được câu hỏi này. Cách làm là nhìn xem những căn có diện tích gần 80 m^2 thường bán bao nhiêu, rồi lấy đó làm mốc. Máy tính cũng làm đúng như vậy, chỉ khác là nó làm bằng một công thức thay vì bằng cảm nhận.

Điều cần chú ý ở đây là thứ chúng ta muốn đoán. Chúng ta muốn đoán một con số, cụ thể là giá tính bằng tỷ đồng, và con số đó có thể là 6.4 hay 7.2 hay bất kỳ giá trị nào ở giữa. Bài toán dự đoán một con số như vậy được gọi là hồi quy (*regression*). Nó khác với bài toán dự đoán một

nhân, ví dụ đoán xem một email là thư rác hay thư bình thường, vì ở đó đáp án chỉ có vài lựa chọn rời rạc. Loại thứ hai gọi là phân loại (*classification*) và sinh viên sẽ học ở bài sau. Cả bài thực hành này chỉ xoay quanh hồi quy.

3.2. Bốn cột trong tệp dữ liệu

Tệp dữ liệu của chúng ta là `data/gia_nha.csv`, gồm 60 dòng và 4 cột. Mỗi dòng là một căn hộ. Mỗi cột là một thông tin về căn hộ đó. Bảng 3 mô tả đầy đủ bốn cột, kèm khoảng giá trị đọc trực tiếp từ tệp.

Bảng 3: Bốn cột của tệp `gia_nha.csv`, với đơn vị và khoảng giá trị thật đọc được từ 60 dòng dữ liệu.

Tên cột	Ý nghĩa	Đơn vị	Khoảng giá trị
<code>dien_tich</code>	Diện tích sàn của căn hộ	m ²	35.5 tới 117.5
<code>so_phong</code>	Số phòng ngủ	phòng	1 tới 4
<code>tuoi_nha</code>	Số năm kể từ khi bàn giao	năm	0 tới 25
<code>gia</code>	Giá bán của căn hộ	tỷ đồng	3.00 tới 9.74

Bốn cột này không đóng vai trò giống nhau. Cột `gia` là thứ chúng ta muốn đoán, nên nó được gọi là biến mục tiêu (*target*). Ba cột còn lại là thứ chúng ta dùng làm căn cứ để đoán, nên chúng được gọi là đặc trưng (*feature*). Cách phân vai này do người làm bài toán quyết định, không phải do dữ liệu tự quy định. Nếu một hôm nào đó sinh viên muốn đoán số phòng từ diện tích và giá thì `so_phong` sẽ thành biến mục tiêu.

Trong phần lớn bài thực hành này, chúng ta cố ý chỉ dùng một đặc trưng duy nhất là `dien_tich`. Lý do rất đơn giản. Với một đặc trưng, mọi thứ vẽ được lên một tờ giấy phẳng và sinh viên nhìn thấy được điều mình đang làm. Tới Mục 8 chúng ta mới dùng cả ba đặc trưng cùng lúc, khi sinh viên đã quen với cách nghĩ.

Còn một điều nữa sinh viên cần biết rõ ngay từ đầu. Bộ dữ liệu này là dữ liệu mô phỏng, được đặt theo mặt bằng giá căn hộ cho gần với thực tế, chứ không phải số liệu giao dịch thật của thị trường. Cách chọn này là có chủ ý. Dữ liệu thị trường thật luôn có ô trống, có giá ghi nhầm, có căn bán gấp rẻ bất thường, và xử lý những chuyện đó là nội dung của một bài khác. Ở bài đầu tiên, sinh viên cần một bộ dữ liệu sạch để tập trung vào ý tưởng của hồi quy. Vì vậy sinh viên đừng lấy các con số trong tài liệu này ra để tư vấn mua bán nhà.

3.3. Đọc dữ liệu bằng pandas

Thư viện pandas là công cụ tiêu chuẩn để đọc và xem dữ liệu dạng bảng trong Python. Tệp `code/b1_doc_du_lieu.py` dưới đây làm năm việc, mỗi việc một lệnh.

```

1  # -*- coding: utf-8 -*-
2  """Bước 1: đọc tệp dữ liệu và nhìn qua một lượt.
3
4  Chú thích trong tệp viết tiếng Việt có dấu, nhưng phần in ra màn hình cố ý
5  viết không dấu. Cửa sổ lệnh Windows thường không hiện được chữ có dấu.
6  """
7
8  import pandas as pd
9
10 # Đọc tệp CSV thành một bảng. Tên df là viết tắt quen dùng của data frame.
11 df = pd.read_csv("data/gia_nha.csv")

```

```

12
13 print("Kích thước bảng (số dòng, số cột):", df.shape)
14 print()
15
16 print("Năm dòng đầu tiên:")
17 print(df.head())
18 print()
19
20 print("Tên các cột:", list(df.columns))
21 print()
22
23 print("Thông kê nhanh cột diện_tích và cột giá:")
24 print(df[["diện_tích", "giá"]].describe().round(2))
25 print()
26
27 print("Số ô bị thiếu trong từng cột:")
28 print(df.isna().sum())

```

Hãy đi qua từng lệnh một.

- `pd.read_csv` đọc tệp CSV có tên ghi trong ngoặc và trả về một bảng dữ liệu, kiểu (*DataFrame*). Chúng ta đặt bảng đó vào biến `df`, là tên viết tắt quen dùng của (*data frame*).
- `df.shape` trả về một cặp số, lần lượt là số dòng và số cột. Đây là cách kiểm tra nhanh nhất để biết tệp đã đọc đúng hay chưa.
- `df.head()` trả về 5 dòng đầu tiên của bảng. Mục đích là nhìn tận mắt vài dòng dữ liệu thật, xem các cột có đúng thứ tự và đúng kiểu số hay không.
- `df.describe()` trả về một bảng thống kê tóm tắt cho từng cột số. Ở đây chúng ta chỉ lấy hai cột `diện_tích` và `giá` cho gọn, và làm tròn hai chữ số bằng `.round(2)`.
- `df.isna().sum()` trả về số ô bị thiếu của từng cột. Lệnh `isna()` đánh dấu đúng hoặc sai cho từng ô, còn `sum()` đếm số ô bị đánh dấu là thiếu.

Chạy tệp này bằng lệnh sau.

```
python code/b1_doc_du_lieu.py
```

Kết quả in ra màn hình đúng như sau.

```

Kích thước bảng (số dòng, số cột): (60, 4)

Năm dòng đầu tiên:
   diện_tích  so_phong  tuoi_nha  giá
0      35.5         1        18  3.00
1      37.6         2         7  3.87
2      49.4         2        20  4.25
3      49.0         2        16  4.60
4      49.9         1        12  3.99

Tên các cột: ['diện_tích', 'so_phong', 'tuoi_nha', 'giá']

Thông kê nhanh cột diện_tích và cột giá:

```

```

      dien_tich    gia
count      60.00  60.00
mean       77.73   6.49
std        20.22   1.64
min        35.50   3.00
25%        63.02   5.16
50%        75.10   6.40
75%        91.88   7.70
max       117.50   9.74

Số ô bị thiếu trong từng cột:
dien_tich    0
so_phong     0
tuoi_nha     0
gia          0
dtype: int64

```

Bây giờ chúng ta đọc kỹ kết quả này. Dòng đầu cho biết bảng có 60 dòng và 4 cột, đúng như mong đợi, nên tệp đã được đọc trọn vẹn. Năm dòng đầu cho thấy các căn nhỏ nhất trong tệp, ví dụ căn 35.5 m² có giá 3.00 tỷ. Quan trọng nhất là dòng cuối cùng: cả bốn cột đều có số ô thiếu bằng 0, nghĩa là không có ô trống nào và chúng ta không phải xử lý dữ liệu khuyết.

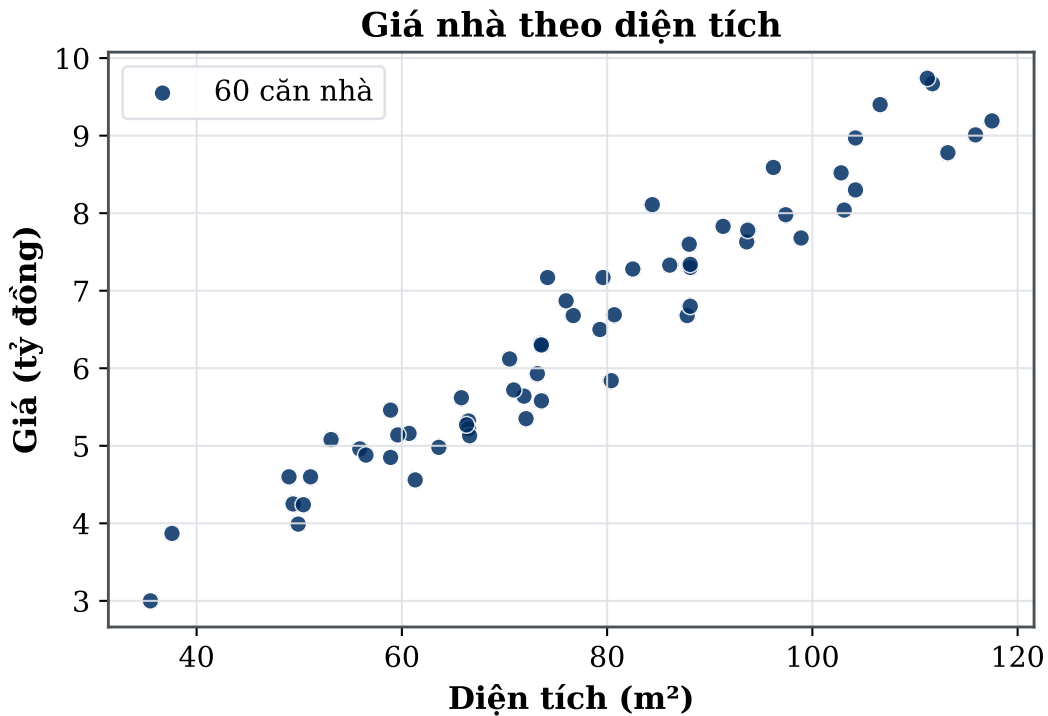
Bảng `describe` có tám dòng, và mỗi dòng có một cái tên đáng nhớ.

- **count** là số ô có giá trị trong cột. Cả hai cột đều bằng 60, khớp với số dòng của bảng.
- **mean** là trung bình cộng. Diện tích trung bình là 77.73 m² và giá trung bình là 6.49 tỷ đồng.
- **std** là độ lệch chuẩn, đo mức các giá trị tản ra xa trung bình. Diện tích có độ lệch chuẩn 20.22 m², nghĩa là phần lớn các căn nằm cách mức 77.73 m² khoảng hai chục mét vuông về mỗi phía.
- **min** là giá trị nhỏ nhất, ở đây là 35.50 m² và 3.00 tỷ đồng.
- **25%** là tứ phân vị dưới. Một phần tư số căn có diện tích nhỏ hơn 63.02 m², và một phần tư số căn có giá dưới 5.16 tỷ.
- **50%** chính là trung vị, tức giá trị đứng giữa khi xếp toàn bộ 60 căn theo thứ tự. Trung vị diện tích là 75.10 m² và trung vị giá là 6.40 tỷ.
- **75%** là tứ phân vị trên. Ba phần tư số căn có diện tích nhỏ hơn 91.88 m² và có giá dưới 7.70 tỷ.
- **max** là giá trị lớn nhất, ở đây là 117.50 m² và 9.74 tỷ đồng.

Có một chi tiết nhỏ mà người làm dữ liệu hay để ý. Trung vị diện tích là 75.10 m², thấp hơn trung bình 77.73 m² khoảng 2.6 m², nghĩa là trong tệp có vài căn rộng kéo trung bình lên. Mức chênh lệch đó rất nhỏ nên đây vẫn là một bộ dữ liệu cân đối.

3.4. Nhìn dữ liệu trước khi tính toán

Đây là thói quen bắt buộc mà sinh viên nên giữ suốt cả môn học: vẽ dữ liệu ra trước khi tính bất cứ thứ gì. Một bảng thống kê chỉ cho vài con số tóm tắt, còn một hình vẽ cho thấy hình dạng của toàn bộ dữ liệu trong một cái liếc mắt. Rất nhiều lỗi nặng trong học máy bị bỏ sót chỉ vì người làm không chịu vẽ.



Hình 1: Sáu mươi căn hộ trong tập dữ liệu, trục hoành là diện tích tính bằng mét vuông và trục tung là giá tính bằng tỷ đồng.

Hình 1 là biểu đồ phân tán (*scatter plot*), trong đó mỗi chấm là một căn hộ, vị trí ngang cho biết diện tích và vị trí dọc cho biết giá. Chúng ta đọc hình này theo ba câu hỏi.

Thứ nhất, các chấm đi theo hướng nào? Chúng đi lên đều đặn từ trái sang phải, nghĩa là căn càng rộng thì giá càng cao, và điều đó khớp với hiểu biết thông thường về nhà đất.

Thứ hai, các chấm nằm theo hình gì? Chúng bám khá sát quanh một đường thẳng tưởng tượng, không cong lên cũng không cong xuống ở hai đầu. Vì vậy một đường thẳng là lựa chọn hợp lý cho bộ dữ liệu này, và đó chính là việc chúng ta sẽ làm ở Mục 4 và Mục 5.

Thứ ba, có chấm nào lạc loài không? Nhìn từ căn nhỏ nhất 35.5 m² tới căn lớn nhất 117.5 m², không có chấm nào bật hẳn ra xa khỏi dải chung. Không có căn nào rộng mà rẻ bất thường, cũng không có căn nào nhỏ mà đắt bất thường, nên chúng ta chưa cần bàn tới chuyện loại bỏ điểm ngoại lai (*outlier*).

Đường thẳng không phải lúc nào cũng đúng

Kết luận ở trên chỉ đúng cho bộ dữ liệu này, vì chúng ta đã nhìn thấy nó bám quanh một đường thẳng. Nếu sinh viên vẽ một bộ dữ liệu khác và thấy các chấm uốn theo hình cong, ví dụ tăng nhanh lúc đầu rồi chững lại, thì một đường thẳng sẽ đoán sai ở cả hai đầu. Nếu các chấm tụ thành nhiều cụm rời nhau, chẳng hạn nhà mặt phố tách hẳn khỏi nhà trong hẻm, thì một đường thẳng duy nhất kéo qua giữa hai cụm sẽ không mô tả đúng cụm nào cả. Trong cả hai trường hợp đó, việc cần làm là đổi mô hình chứ không phải cố chỉnh đường thẳng cho khéo hơn. Hình vẽ chính là thứ báo cho sinh viên biết lúc nào phải đổi.

Ghi nhớ

Hãy chạy `df.isna().sum()` với mọi bộ dữ liệu mới, kể cả khi sinh viên tin rằng nó sạch. Bộ dữ liệu trong bài này có số ô thiếu bằng 0 ở cả bốn cột, nhưng đó là vì nó đã được chuẩn

bị sẵn cho lớp học. Dữ liệu thu thập ngoài đời gần như luôn thiếu ô, do người nhập bỏ trống, do cảm biến hỏng, hoặc do ghép hai nguồn có số cột khác nhau. Nếu bỏ qua bước kiểm tra này, phép tính trung bình vẫn chạy nhưng kết quả sẽ lệch, và sinh viên sẽ rất khó tìm ra nguyên nhân về sau.

4. Hồi quy tuyến tính là gì

Ở mục trước sinh viên đã nhìn thấy 60 căn hộ nằm rải rác trên đồ thị. Đám điểm đó đi lên từ trái sang phải, nghĩa là căn càng rộng thì giá càng cao. Bây giờ ta hỏi một câu cụ thể hơn nhiều. Có một căn 80 m² không nằm trong bảng, vậy ta đoán giá của nó là bao nhiêu? Muốn trả lời thì phải có một quy tắc tính giá từ diện tích. Quy tắc đó chính là thứ mà trong học máy người ta gọi là mô hình (*model*). Mục này dựng lên mô hình đơn giản nhất có thể, rồi dựng tiếp một thứ còn quan trọng hơn, đó là cách chấm điểm cho mô hình.

4.1. Ý tưởng chỉ gói trong một đường thẳng

Sinh viên hãy tưởng tượng mình cầm một cây thước đặt lên đám 60 điểm ở mục trước. Sinh viên xoay thước cho nghiêng vừa phải, rồi đẩy nó lên hoặc xuống, sao cho thước đi xuyên qua giữa đám điểm. Cây thước đặt xong chính là mô hình của chúng ta. Nói gọn lại, hồi quy tuyến tính (*linear regression*) chỉ là việc chọn một đường thẳng chạy xuyên qua dữ liệu. Cả mô hình vì thế gói trọn trong hai con số, một con số tả độ nghiêng của thước và một con số tả chỗ thước cắt trục dọc.

Mô hình hồi quy tuyến tính một biến

$$\hat{y} = w \cdot x + b$$

Bốn ký hiệu trong công thức trên có nghĩa như sau.

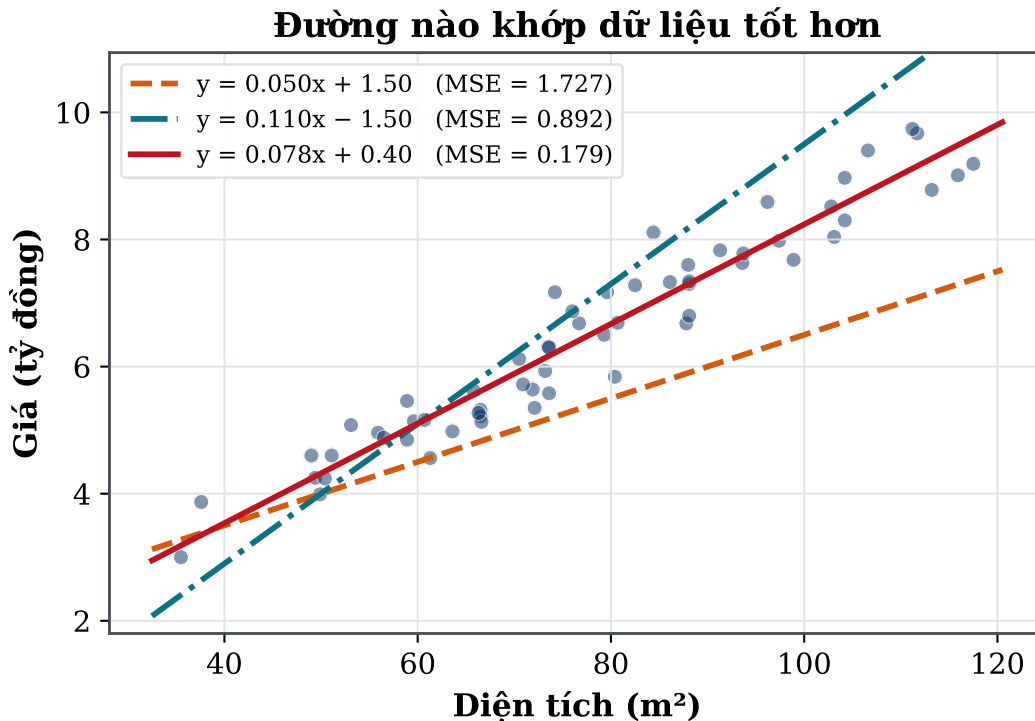
- x là con số ta đưa vào, ở bài này là diện tích căn hộ tính bằng mét vuông.
- $\hat{y}$ là giá mà mô hình đoán ra cho căn hộ đó, tính bằng tỷ đồng.
- w là hệ số góc (*slope*), tức độ nghiêng của đường thẳng.
- b là hệ số chặn (*intercept*), tức chỗ đường thẳng cắt trục tung.

Hai ký hiệu w và b nghe khô khan nhưng chúng trả lời hai câu hỏi rất đời thường. Hệ số góc w trả lời câu hỏi thứ nhất, thêm một mét vuông thì giá tăng thêm bao nhiêu tỷ đồng. Nếu w bằng 0.08 thì mỗi mét vuông tăng thêm kéo theo giá tăng thêm 0.08 tỷ, tức 80 triệu đồng. Mục 5 sẽ tính ra con số w đúng cho bộ dữ liệu này, còn bây giờ sinh viên chỉ cần nhớ w đo mức tăng giá trên một mét vuông. Hệ số chặn b trả lời câu hỏi thứ hai, đường thẳng cắt trục tung ở độ cao nào. Về mặt hình học, b là giá trị của đường thẳng khi x bằng 0, nên nó quyết định đường thẳng nằm cao hay nằm thấp trên đồ thị. Đừng cố đọc b thành giá của một căn hộ rộng 0 m², vì căn hộ như vậy không tồn tại và bộ dữ liệu của ta bắt đầu từ 35.5 m².

Còn một chi tiết nhỏ mà nếu lẫn thì về sau sinh viên sẽ rối, đó là dấu mũ trên đầu chữ y . Ký hiệu $\hat{y}$, đọc là y mũ, luôn là giá mà mô hình ĐOÁN. Ký hiệu y trần, không mũ, luôn là giá THẬT ghi trong tệp `gia_nha.csv`. Hai con số này gần như không bao giờ bằng nhau, và toàn bộ phần còn lại của mục 4 chỉ xoay quanh khoảng cách giữa chúng.

4.2. Ba đường thẳng, đường nào đúng

Có công thức rồi thì vẽ đường thẳng nào cũng được, chỉ cần chọn hai số w và b là xong. Cái khó nằm ở chỗ chọn cặp số nào mới phải. Hình 2 đặt ba đường thẳng khác nhau lên đúng 60 căn hộ đó.

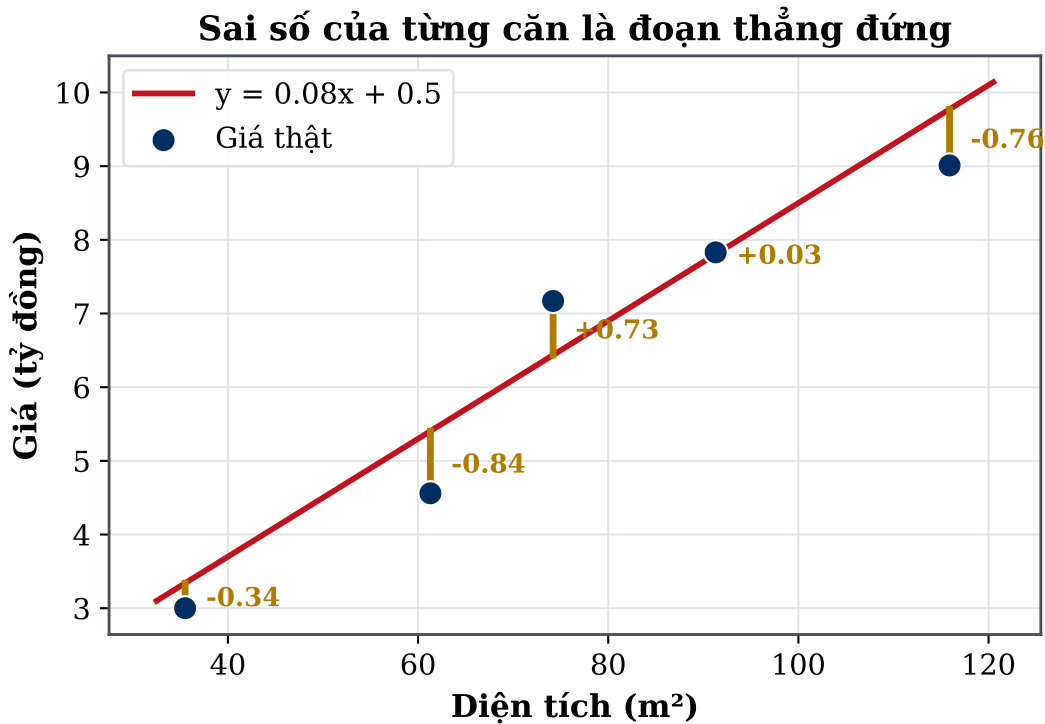


Hình 2: Ba đường thẳng ứng viên vẽ trên cùng 60 căn hộ, mỗi đường kèm một con số trong phần chú giải.

Trước khi đọc tiếp, sinh viên hãy nhìn hình chừng nửa phút rồi tự chọn lấy một đường mà mình cho là hợp nhất. Trong lớp, câu hỏi này thường nhận được cả ba đáp án khác nhau, và ai cũng có lý lẽ riêng. Người thì bảo đường này đi sát mấy căn nhỏ hơn, người thì bảo đường kia bám mấy căn lớn tốt hơn. Tranh luận như vậy không bao giờ ngã ngũ, vì mắt người mỗi lúc nhìn một khác. Do đó ta cần một con số, một thước đo chấm điểm từng đường thẳng để đem ra so. Con số đó đã in sẵn trong chú giải của hình, nhưng sinh viên hãy tạm bỏ qua nó. Hai tiểu mục tiếp theo sẽ dựng con số đó lên từ đầu, và khi quay lại hình này sinh viên sẽ đọc được nó.

4.3. Đo sai số của một căn nhà

Ta bắt đầu từ một căn nhà duy nhất cho thật dễ. Với một căn cụ thể, ta có giá thật y và có giá mô hình đoán $\hat{y}$. Sai số của căn đó được định nghĩa là giá thật trừ đi giá đoán, viết gọn là $y - \hat{y}$. Đây chỉ là một phép trừ, không có gì bí ẩn cả.



Hình 3: Năm căn hộ và đường thẳng thử $y = 0.08x + 0.5$, mỗi đoạn thẳng đứng màu vàng là sai số của một căn.

Hình 3 cho sinh viên nhìn thấy phép trừ đó bằng mắt. Mỗi chấm xanh là giá thật của một căn, còn đường màu đỏ là mô hình đang thử. Đoạn thẳng đứng màu vàng nối chấm xanh với đường đỏ, và độ dài của nó chính là sai số của căn đó. Đoạn vàng càng dài thì mô hình càng trượt xa, đoạn vàng ngắn tũn nghĩa là đoán khá trúng. Bên cạnh mỗi đoạn có ghi sẵn giá trị sai số kèm dấu.

Dấu của sai số mang một thông tin riêng nên sinh viên đừng vứt nó đi. Sai số dương nghĩa là giá thật cao hơn giá đoán, tức mô hình đoán hụt. Sai số âm nghĩa là giá thật thấp hơn giá đoán, tức mô hình đoán cao hơn thực tế. Lấy ngay hai căn trong hình làm ví dụ cho rõ. Căn 74.2 m² có giá thật 7.17 tỷ, mô hình đoán 6.44 tỷ, sai số là +0.73, nên mô hình đoán hụt 0.73 tỷ. Căn 115.9 m² có giá thật 9.01 tỷ, mô hình đoán 9.77 tỷ, sai số là -0.76, nên mô hình hô giá cao hơn thực tế 0.76 tỷ.

4.4. Từ sai số của từng căn tới một con số duy nhất

Giờ ta có 60 sai số, mỗi căn một cái, nhưng ta cần đúng một con số để chấm điểm cả đường thẳng. Phản xạ đầu tiên của hầu hết sinh viên là cộng thẳng 60 sai số đó lại. Cách này hỏng, và nó hỏng theo kiểu nguy hiểm nhất, đó là hỏng mà vẫn ra con số đẹp. Lý do nằm ở dấu âm dấu dương. Ở bảng của tiểu mục sau, đường thẳng thứ nhất trượt +1.96 tỷ tại căn 74.2 m². Giả sử có thêm một căn bị trượt -1.96 tỷ. Cộng thẳng hai sai số này lại thì được 0, trong khi thực tế mô hình đã đoán sai nặng ở cả hai căn. Một mô hình tồi vì vậy có thể trông như hoàn hảo, chỉ nhờ các sai số triệt tiêu lẫn nhau.

Muốn chữa thì phải bỏ dấu đi trước khi cộng, và có hai cách quen thuộc. Cách thứ nhất là lấy trị tuyệt đối của từng sai số. Cách thứ hai là bình phương từng sai số, vì bình phương của số âm cũng thành số dương. Ở bài này ta chọn bình phương, vì hai lý do. Lý do thứ nhất là bình phương phạt rất nặng những cú trượt lớn. Trượt 1 tỷ thì bị tính thành 1, nhưng trượt 2 tỷ thì bị tính thành 4, tức nặng gấp bốn lần chứ không phải gấp đôi. Điều đó đúng với mong muốn của ta, vì một mô hình đoán lệch 2 tỷ ở một căn thì tệ hơn nhiều so với đoán lệch 1 tỷ ở hai căn. Lý

do thứ hai mang tính kỹ thuật nhưng rất đáng giá. Hàm bình phương là hàm trơn, không có chỗ gãy góc như hàm trị tuyệt đối, nên ta tìm được điểm thấp nhất của nó bằng một công thức đóng. Nhờ vậy mục 5 sẽ giải ra w và b chỉ bằng vài phép tính, không cần dò dẫm.

Sai số toàn phương trung bình

$$\text{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2$$

Công thức trên đọc từ trong ra ngoài thì rất nhẹ nhàng. Chữ n là số căn nhà ta đang tính, ở bộ dữ liệu đầy đủ thì n bằng 60. Chỉ số i chạy từ căn thứ nhất tới căn thứ n , nên x_i là diện tích căn thứ i và y_i là giá thật của căn thứ i . Phần $w x_i + b$ là giá mô hình đoán cho căn thứ i , còn cả cụm trong ngoặc là sai số của căn đó. Dấu mũ 2 ở ngoài ngoặc là bước bình phương để bỏ dấu. Dấu $\sum$ nghĩa là cộng tất cả lại, và $\frac{1}{n}$ ở đầu là bước chia cho số căn để lấy trung bình. Nhờ bước lấy trung bình này mà ta so được MSE của 5 căn với MSE của 60 căn mà không bị số lượng căn làm nhiễu. Tên đầy đủ của nó là sai số toàn phương trung bình (*mean squared error*), và hầu như mọi tài liệu đều gọi tắt là MSE. Cuối cùng, hãy để ý MSE được viết là $\text{MSE}(w, b)$, nghĩa là nó phụ thuộc vào w và b . Đổi đường thẳng thì MSE đổi theo, và đó chính là chỗ bấm để ta đi tìm đường thẳng tốt nhất.

4.5. Tính thử bằng tay trên năm căn nhà

Lý thuyết tới đây là đủ, ta cho máy tính thử ngay. Đoạn mã dưới đây lấy năm căn trải từ 35.5 m² tới 115.9 m² cho dễ nhìn, rồi chấm điểm hai đường thẳng khác nhau trên đúng năm căn đó.

```

1  #-*- coding: utf-8 -*-
2  """Bước 2: đo xem một đường thẳng dự đoán sai bao nhiêu.
3
4  Chỉ lấy 5 căn cho dễ nhìn. Thử hai đường thẳng khác nhau rồi so sai số
5  trung bình bình phương của từng đường.
6  """
7
8  import pandas as pd
9
10 df = pd.read_csv("data/gia_nha.csv")
11
12 # Lấy 5 căn trải đều từ nhỏ nhất tới lớn nhất, không lấy 5 căn đầu bảng
13 # vì chúng có diện tích gần bằng nhau nên nhìn không rõ.
14 nho = df.iloc[[0, 14, 29, 44, 59]]
15
16 x = nho["dien_tich"].to_numpy()
17 y = nho["gia"].to_numpy()
18
19
20 def mse(w, b):
21     """Sai số trung bình bình phương của đường thẳng y = w*x + b."""
22     y_du_doan = w * x + b
23     sai_so = y - y_du_doan
24     return (sai_so ** 2).mean()

```

```

25
26
27 print("Nam can duoc chon:")
28 for i in range(5):
29     print(f" {x[i]:6.1f} m2 -> {y[i]:5.2f} ty")
30 print()
31
32 for w, b in [(0.05, 1.5), (0.08, 0.5)]:
33     print(f"Duong thang y = {w} * x + {b}")
34     for i in range(5):
35         du_doan = w * x[i] + b
36         sai_so = y[i] - du_doan
37         print(f" x = {x[i]:6.1f}    that = {y[i]:5.2f}"
38               f"    du doan = {du_doan:5.2f}    sai so = {sai_so:+6.2f}")
39     print(f" MSE = {mse(w, b):.4f}")
40     print()
41
42 print("Duong nao co MSE nho hon thi duong do khop du lieu tot hon.")

```

Hàm `mse` trong mã chỉ làm đúng ba việc theo thứ tự trong công thức. Nó tính giá đoán, lấy giá thật trừ giá đoán, rồi bình phương và lấy trung bình bằng `.mean()`. Chạy tệp này ta được kết quả sau.

```

Nam can duoc chon:
 35.5 m2 ->  3.00 ty
 61.3 m2 ->  4.56 ty
 74.2 m2 ->  7.17 ty
 91.3 m2 ->  7.83 ty
115.9 m2 ->  9.01 ty

Duong thang y = 0.05 * x + 1.5
x =  35.5    that =  3.00    du doan =  3.28    sai so =  -0.28
x =  61.3    that =  4.56    du doan =  4.56    sai so =  -0.00
x =  74.2    that =  7.17    du doan =  5.21    sai so =  +1.96
x =  91.3    that =  7.83    du doan =  6.07    sai so =  +1.76
x = 115.9    that =  9.01    du doan =  7.30    sai so =  +1.71
MSE = 1.9947

Duong thang y = 0.08 * x + 0.5
x =  35.5    that =  3.00    du doan =  3.34    sai so =  -0.34
x =  61.3    that =  4.56    du doan =  5.40    sai so =  -0.84
x =  74.2    that =  7.17    du doan =  6.44    sai so =  +0.73
x =  91.3    that =  7.83    du doan =  7.80    sai so =  +0.03
x = 115.9    that =  9.01    du doan =  9.77    sai so =  -0.76
MSE = 0.3896

Duong nao co MSE nho hon thi duong do khop du lieu tot hon.

```

Ta đọc kết quả này cho kỹ, vì nó gói gọn toàn bộ ý tưởng của mục 4. Đường thẳng thứ nhất

là $y = 0.05x + 1.5$ và nó đạt MSE bằng 1.9947. Đường thẳng thứ hai là $y = 0.08x + 0.5$ và nó đạt MSE bằng 0.3896. Số nhỏ hơn là số tốt hơn, vì MSE đo mức trượt chứ không đo mức giới. Vậy đường thứ hai thắng, và thắng đậm, vì 1.9947 lớn hơn 0.3896 tới hơn năm lần.

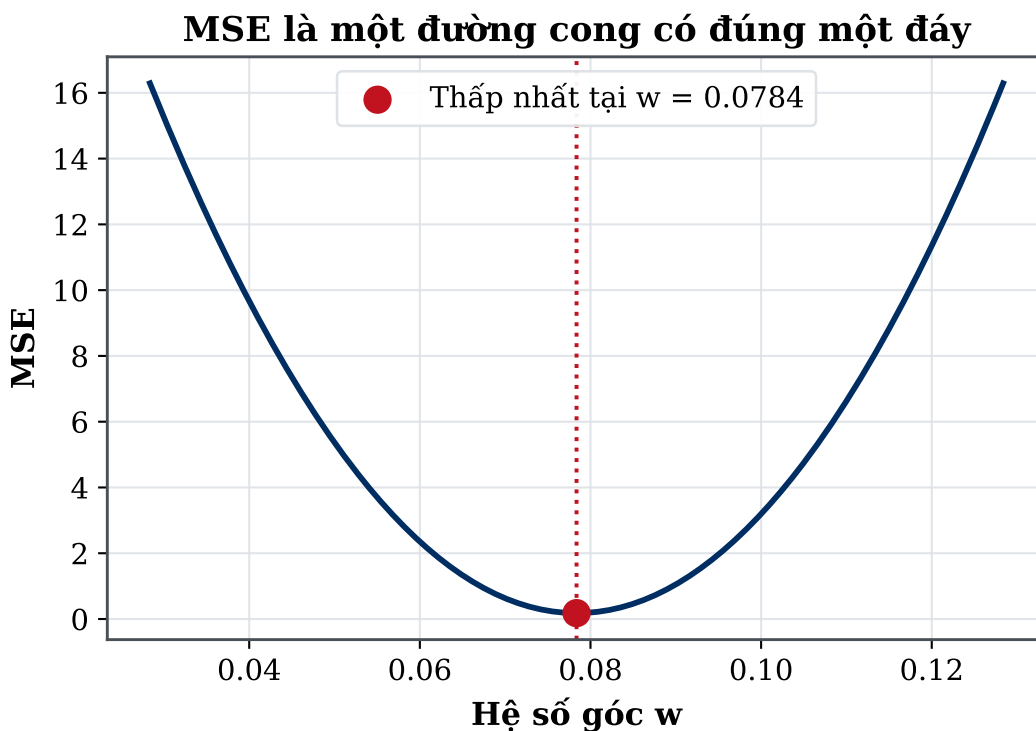
Bây giờ mời sinh viên nhìn kỹ căn 61.3 m² ở cả hai bảng. Đường thứ nhất đoán căn này là 4.56 tỷ trong khi giá thật cũng là 4.56 tỷ, sai số -0.00 , gần như trúng phóc. Đường thứ hai lại đoán căn này là 5.40 tỷ, sai số -0.84 , tức trượt khá xa. Nếu chỉ chấm điểm bằng căn 61.3 m² thì ta sẽ kết luận ngược hẳn và chọn nhầm đường thứ nhất. Nhưng nhìn cả năm căn thì đường thứ nhất trượt tới $+1.96$, $+1.76$ và $+1.71$ ở ba căn lớn, nên tổng thể nó vẫn tệ hơn nhiều. Bài học rút ra ngắn gọn thế này, một mô hình không được chấm điểm bằng một trường hợp lẻ, mà phải chấm trên toàn bộ dữ liệu. Đây là lỗi thường gặp nhất của người mới, và sinh viên sẽ còn gặp lại nó ở mục 6 dưới một hình thức khác.

4.6. Bài toán thật sự cần giải

Đến đây ta phát biểu được bài toán bằng lời, rõ ràng và không còn mơ hồ. Trong vô số cặp w và b có thể chọn, hãy tìm cặp làm cho MSE nhỏ nhất trên bộ dữ liệu đang có. Cặp số đó cho ra đường thẳng mà ta gọi là đường hồi quy tốt nhất. Học máy, ở mức căn bản nhất, chính là việc đó và chỉ việc đó.

Có thước đo rồi thì ta quay lại Hình 2 và đọc được ngay các con số trong chú giải. Đường $y = 0.050x + 1.50$ có MSE bằng 1.727, đường $y = 0.110x - 1.50$ có MSE bằng 0.892, còn đường màu đỏ có MSE bằng 0.179. Đường màu đỏ thắng, và ở mục 5 sinh viên sẽ thấy nó không phải do ai đoán mò mà ra. Sinh viên để ý một điều nhỏ để khỏi thắc mắc về sau. Ba con số vừa đọc được tính trên cả 60 căn, còn hai con số 1.9947 và 0.3896 ở tiểu mục trước chỉ tính trên 5 căn, nên chúng không việc gì phải bằng nhau.

Câu hỏi còn lại là tìm cặp w và b tốt nhất bằng cách nào. Hình 4 cho ta một manh mối quan trọng.



Hình 4: Giữ nguyên hệ số chặn và cho hệ số góc chạy, đường cong MSE có đúng một đáy duy nhất.

Cách vẽ hình này như sau. Ta giữ nguyên hệ số chặn b ở đúng giá trị tốt nhất của nó, chỉ cho

hệ số góc w chạy dần từ nhỏ tới lớn. Phải nói rõ điều kiện đó, vì nếu cố định b ở một giá trị tùy tiện thì đáy của đường cong sẽ nằm ở chỗ khác. Ứng với mỗi giá trị w , ta tính MSE trên cả 60 căn rồi chấm một điểm. Nối các điểm đó lại thì được đường cong trong hình, trục hoành là w và trục tung là MSE. Đường cong này có dạng lòng chảo, hai bên cao và ở giữa võng xuống. Chỗ võng sâu nhất nằm tại w bằng 0.0784, và đó chính là hệ số góc tốt nhất mà ta đang đi tìm. Điều đáng mừng là lòng chảo này chỉ có đúng một đáy. Cứ tưởng tượng sinh viên thả một hòn bi vào lòng chảo, dù thả ở đâu thì nó cũng lăn về đúng một chỗ. Nhiều bài toán học máy khác có đồ thị lồi lõm với nhiều đáy giả (*local minimum*), nên máy dễ mắc kẹt ở một đáy nông và tưởng nhầm là đã xong. Hồi quy tuyến tính không bị chuyện đó, và chính nhờ vậy mục 5 giải được bài toán bằng một công thức, còn mục 7 dò được lời giải bằng một vòng lặp đơn giản.

MSE có đơn vị khó hình dung

Giá nhà tính bằng tỷ đồng, mà MSE lại bình phương sai số lên, nên đơn vị của MSE là tỷ đồng bình phương. Không ai đi mua nhà mà nói “tôi lệch mất ba tỷ đồng bình phương” cả. Vì vậy bản thân con số MSE gần như không mang ý nghĩa đời thường, nó chỉ hữu ích khi đem so mô hình này với mô hình kia. Mục 6 sẽ giới thiệu RMSE, tức căn bậc hai của MSE, để kéo sai số về lại đúng đơn vị tỷ đồng cho sinh viên dễ cảm nhận.

5. Tìm đường thẳng tốt nhất

Ở mục trước sinh viên đã biết thế nào là một đường thẳng tốt. Đường thẳng tốt là đường có sai số toàn phương trung bình nhỏ nhất. Câu hỏi còn lại là làm sao tìm được nó. Thử tay từng đường một như ở mục trước thì không bao giờ xong, vì có vô số đường thẳng. Có hai cách làm đúng đắn, và trong bài này sinh viên sẽ làm cả hai. Cách thứ nhất là dùng công thức mà toán học đã giải sẵn. Cách thứ hai là gọi một thư viện viết sẵn, đó là cách người đi làm thật sự dùng. Hai cách phải cho ra cùng một kết quả, và lát nữa sinh viên sẽ tự kiểm chứng điều đó.

5.1. Cách 1: dùng công thức có sẵn

Khi mô hình chỉ có một biến đầu vào là diện tích, bài toán tìm w và b nhỏ gọn tới mức toán học giải ra được bằng giấy bút. Người ta gọi lời giải đó là nghiệm đóng (*closed form*). Nghĩa là ta không phải dò, không phải thử, không phải lặp. Ta thay số vào công thức và lấy ngay kết quả. Trước khi viết công thức ra, thầy giải thích ý của nó bằng lời để sinh viên không phải học thuộc.

Ý thứ nhất nằm ở tử số. Tử số cộng dồn tích của hai độ lệch. Độ lệch thứ nhất là diện tích của một căn trừ đi diện tích trung bình. Độ lệch thứ hai là giá của chính căn đó trừ đi giá trung bình. Nếu căn nào rộng hơn mức trung bình mà cũng đắt hơn mức trung bình thì hai độ lệch cùng dương, nên tích của chúng dương. Nếu căn nào hẹp hơn mức trung bình mà cũng rẻ hơn mức trung bình thì hai độ lệch cùng âm, nên tích của chúng vẫn dương. Vì vậy tử số lớn khi diện tích và giá đi cùng chiều với nhau trên toàn bộ dữ liệu.

Ý thứ hai nằm ở mẫu số. Mẫu số chỉ nhìn vào diện tích, nó cộng dồn bình phương độ lệch của diện tích so với trung bình. Nó đo độ phân tán của diện tích, tức là dữ liệu trải rộng hay co cụm. Chia tử số cho mẫu số chính là quy đổi cái xu hướng đi cùng chiều đó về đơn vị tỷ đồng trên một mét vuông.

Công thức cần nhớ

$$w = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad b = \bar{y} - w\bar{x}$$

Trong công thức trên, n là số căn hộ, ở đây $n = 60$. Ký hiệu x_i là diện tích của căn thứ i , còn y_i là giá của chính căn đó. Ký hiệu $\bar{x}$ đọc là x gạch, nó là diện tích trung bình của cả 60 căn, bằng 77.7317 m². Ký hiệu $\bar{y}$ đọc là y gạch, nó là giá trung bình của cả 60 căn, bằng 6.4933 tỷ đồng. Dấu $\sum$ là dấu tổng, nó bảo ta chạy hết 60 căn rồi cộng tất cả lại.

Công thức của b đáng để sinh viên nhìn kỹ một lát. Chuyển về nó thành $\bar{y} = w\bar{x} + b$, và đó đúng là phương trình đường thẳng tại điểm có hoành độ $\bar{x}$. Nói cách khác, đường thẳng tốt nhất luôn đi qua điểm trung bình của đám dữ liệu. Điểm đó ở đây là căn hộ tương đương rộng 77.7317 m² và giá 6.4933 tỷ đồng. Đây là một tính chất đẹp và rất dễ nhớ, vì nó có nghĩa là đường thẳng luôn đi xuyên qua giữa đám mây điểm chứ không lệch hẳn sang một bên.

Bây giờ ta viết công thức đó thành mã. Hãy mở tệp `code/b3_cong_thuc.py` và đọc theo thầy.

```

1  # -*- coding: utf-8 -*-
2  """Bước 3: tìm w và b tốt nhất bằng công thức, không cần thư viện."""
3
4  import numpy as np
5  import pandas as pd
6
7  df = pd.read_csv("data/gia_nha.csv")
8  x = df["dien_tich"].to_numpy()
9  y = df["gia"].to_numpy()
10
11  x_tb = x.mean()
12  y_tb = y.mean()
13
14  # Công thức bình phương tối thiểu cho đường thẳng một biến
15  tu_so = ((x - x_tb) * (y - y_tb)).sum()
16  mau_so = ((x - x_tb) ** 2).sum()
17
18  w = tu_so / mau_so
19  b = y_tb - w * x_tb
20
21  print(f"Trung bình diện tích : {x_tb:.4f}")
22  print(f"Trung bình gia      : {y_tb:.4f}")
23  print(f"Tu so                : {tu_so:.4f}")
24  print(f"Mau so                : {mau_so:.4f}")
25  print()
26  print(f"He so goc          w = {w:.6f}")
27  print(f"He so chan         b = {b:.6f}")
28  print()
29  print(f"Mo hình: gia = {w:.4f} * dien_tich + {b:.4f}")
30  print()
31
32  y_du_doan = w * x + b
33  mse = ((y - y_du_doan) ** 2).mean()
34  print(f"MSE tren toan bo 60 can: {mse:.4f}")
35
36  # Dự đoán cho một căn 80 m2
37  print(f"Can 80 m2 duoc du doan: {w * 80 + b:.3f} ty dong")

```

Điều đáng chú ý nhất trong đoạn mã này là không hề có vòng lặp nào. Trong công thức có dấu tổng chạy qua 60 căn, vậy mà mã Python lại không có dòng `for` nào. Lý do là numpy làm việc trên cả mảng một lúc. Khi sinh viên viết $x - x_{tb}$, numpy hiểu rằng phải lấy từng phần tử của mảng x trừ đi con số x_{tb} , rồi trả về một mảng mới cũng có 60 phần tử. Tương tự, phép nhân $(x - x_{tb}) * (y - y_{tb})$ nhân từng cặp phần tử tương ứng với nhau, phần tử thứ nhất nhân phần tử thứ nhất, phần tử thứ hai nhân phần tử thứ hai, và cứ thế. Cuối cùng `.sum()` cộng cả 60 số đó lại thành một con số duy nhất, đúng bằng dấu tổng trong công thức. Cách viết này gọn hơn vòng lặp, và trên dữ liệu lớn nó còn chạy nhanh hơn nhiều lần vì numpy tính bằng mã máy chứ không qua từng bước của Python.

Chạy tệp và sinh viên sẽ thấy kết quả sau.

```
Trung bình diện tích : 77.7317
Trung bình gia       : 6.4933
Tu so                : 1890.2297
Mau so               : 24120.2898

He so goc           w = 0.078367
He so chan           b = 0.401752

Mo hinh: gia = 0.0784 * dien_tich + 0.4018

MSE tren toan bo 60 can: 0.1790
Can 80 m2 duoc du doan: 6.671 ty dong
```

Ta đọc kết quả này từng dòng một. Từ số bằng 1890.2297 và mẫu số bằng 24120.2898, chia ra được hệ số góc $w = 0.078367$. Hệ số chặn là $b = 0.401752$, lấy từ 6.4933 trừ đi tích của 0.078367 với 77.7317. Đơn vị của w là tỷ đồng trên một mét vuông, nên 0.078367 tỷ đổi ra là khoảng 78.4 triệu đồng. Đọc bằng lời thì mô hình đang nói rằng mỗi mét vuông ở khu vực này đáng giá khoảng 78 triệu đồng. Một căn rộng hơn căn khác 10 m² thì theo mô hình sẽ đắt hơn khoảng 0.78 tỷ đồng. Con số này dễ kiểm tra bằng cảm nhận thực tế, và đó là một thói quen tốt mà thầy mong sinh viên giữ suốt môn học. Sai số toàn phương trung bình của đường này trên cả 60 căn là 0.1790. Không một đường thẳng nào khác đạt được con số nhỏ hơn 0.1790 trên bộ dữ liệu này, vì công thức đã tìm đúng đáy của đường cong sai số mà sinh viên đã thấy ở mục trước. Dòng cuối cùng là phần vui nhất, mô hình dự đoán một căn 80 m² có giá 6.671 tỷ đồng.

Đừng hiểu sai hệ số chặn

Nhìn vào $b = 0.401752$ rất dễ kết luận rằng một căn hộ rộng 0 m² thì giá 0.4 tỷ đồng. Kết luận đó vô nghĩa, vì không có căn hộ nào rộng 0 m². Quan trọng hơn, trong 60 căn của ta căn nhỏ nhất đã là 35.5 m², nên mô hình chưa từng nhìn thấy căn nào nhỏ hơn con số đó. Đường thẳng kéo dài về phía 0 chỉ là phần mở rộng trên giấy, nó không dựa trên dữ liệu nào cả. Hãy hiểu b đơn giản là con số giúp đường thẳng nằm đúng độ cao, nhờ nó mà đường thẳng đi qua giữa đám điểm thay vì nằm hệt bên dưới.

5.2. Cách 2: để scikit-learn làm

Công thức ở trên rất đáng hiểu, nhưng trong công việc thật thì không ai gõ lại nó. Lý do thứ nhất là gõ tay dễ sai dấu. Lý do thứ hai là công thức đó chỉ đúng cho trường hợp một biến, còn ở phần cuối bài ta sẽ dùng ba biến cùng lúc và dạng viết tay sẽ dài ra rất nhanh. Lý do thứ ba là thư viện

đã được hàng triệu người dùng và kiểm tra, nên nó đáng tin hơn đoạn mã ta vừa viết trong mười phút. Thư viện chuẩn cho việc này là scikit-learn.

```

1  # -*- coding: utf-8 -*-
2  """Bước 4: làm lại đúng việc đó bằng scikit-learn, chỉ mất ba dòng."""
3
4  import pandas as pd
5  from sklearn.linear_model import LinearRegression
6
7  df = pd.read_csv("data/gia_nha.csv")
8
9  # X phải là bảng hai chiều, y là dãy một chiều.
10 # Chú ý X viết hai cặp ngoặc vuông, còn y chỉ một cặp.
11 X = df[["dien_tich"]]
12 y = df["gia"]
13
14 mo_hinh = LinearRegression()
15 mo_hinh.fit(X, y)
16
17 print("He so goc w =", round(float(mo_hinh.coef_[0]), 6))
18 print("He so chan b =", round(float(mo_hinh.intercept_), 6))
19 print()
20
21 # So sánh với kết quả tính tay ở bước 3
22 print("Ket qua tinh tay o buoc 3: w = 0.078367, b = 0.401752")
23 print()
24
25 can_moi = pd.DataFrame({"dien_tich": [80.0, 100.0]})
26 du_doan = mo_hinh.predict(can_moi)
27 for dt, gia in zip(can_moi["dien_tich"], du_doan):
28     print(f"Can {dt:.0f} m2 -> du doan {gia:.3f} ty dong")

```

Đoạn mã trên ngắn tới bất ngờ, và phần cốt lõi chỉ nằm ở ba dòng. Ba dòng đó theo một khuôn mẫu cố định mà sinh viên sẽ gặp lại suốt cả môn học.

Bảng 4: Ba bước chuẩn của mọi mô hình trong scikit-learn, minh họa bằng hồi quy tuyến tính.

Bước	Dòng mã	Máy làm gì
1	<code>mo_hinh = LinearRegression()</code>	Tạo ra một mô hình rỗng, chưa học gì cả, w và b chưa có giá trị.
2	<code>mo_hinh.fit(X, y)</code>	Học từ dữ liệu, tức là tìm w và b làm sai số nhỏ nhất.
3	<code>mo_hinh.predict(can_moi)</code>	Dùng w và b vừa học để đoán giá cho những căn mới.

Điều thầy muốn sinh viên nhớ kỹ là ba bước này không riêng của hồi quy tuyến tính. Mọi mô hình trong scikit-learn đều theo đúng khuôn mẫu đó, từ cây quyết định (*decision tree*) tới máy vector tựa (*support vector machine*). Muốn đổi sang mô hình khác, sinh viên chỉ thay tên lớp ở bước 1, còn hai bước sau giữ nguyên từng chữ. Học một lần là dùng được cho mọi mô hình về sau, nên đây là mười phút đáng giá nhất của buổi hôm nay.

Hình dạng dữ liệu, chỗ sai kinh điển

Hãy nhìn thật kỹ hai dòng tạo dữ liệu đầu vào trong đoạn mã trên. Biến x viết là `df[["dien_tich"]]` với HAI cặp ngoặc vuông, còn biến y viết là `df["gia"]` với MỘT cặp. Sự khác nhau này có lý do rõ ràng. Scikit-learn quy ước rằng x là một bảng hai chiều, mỗi dòng là một căn hộ và mỗi cột là một đặc trưng (*feature*). Hôm nay ta chỉ có một cột là `dien_tich`, nhưng nó vẫn phải nằm trong một cái bảng, và hai cặp ngoặc vuông chính là cách bảo pandas trả về một bảng. Ngược lại y là đáp án cần đoán, mỗi căn một con số, nên nó chỉ là một dãy một chiều và dùng một cặp ngoặc là đủ. Nếu sinh viên viết nhầm một cặp ngoặc cho x thì chương trình sẽ dừng với thông báo `Expected a 2-dimensional container but got pandas.Series instead`. Gặp dòng chữ đó thì đừng hoảng, hãy quay lại đếm số cặp ngoặc vuông ở x .

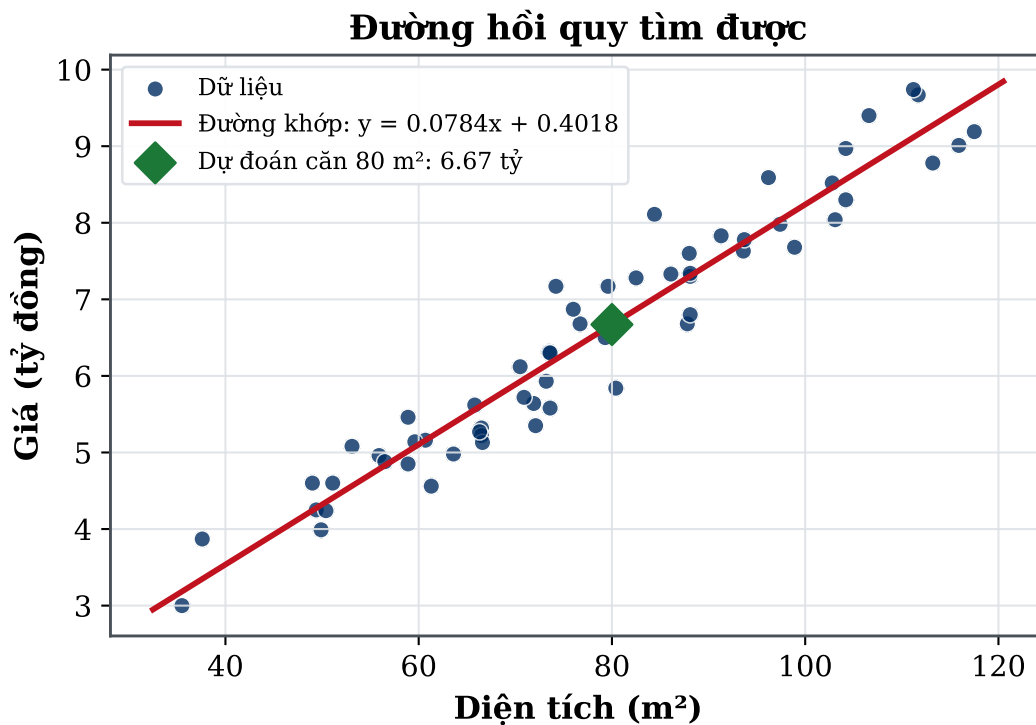
Bây giờ chạy tệp code/`b4_sklearn.py` và so kết quả với cách tính tay.

```
He so goc w = 0.078367
He so chan b = 0.401752

Ket qua tinh tay o buoc 3: w = 0.078367, b = 0.401752

Can 80 m2 -> du doan 6.671 ty dong
Can 100 m2 -> du doan 8.238 ty dong
```

Scikit-learn cho $w = 0.078367$ và $b = 0.401752$. Đó là đúng hai con số ta đã tự tính ra ở cách 1, giống nhau tới từng chữ số được in ra. Đây không phải là trùng hợp may mắn. Cả hai cách đều đang giải cùng một bài toán, đó là tìm đường thẳng có sai số toàn phương trung bình nhỏ nhất. Bài toán đó chỉ có duy nhất một lời giải, vì đường cong sai số của bài toán này chỉ có đúng một đáy. Cho nên ai giải đúng thì cũng phải ra cùng một chỗ, dù người giải bằng công thức viết tay còn người giải bằng thư viện. Hai dòng cuối cho thấy công dụng thật sự của mô hình, một căn 80 m² được đoán 6.671 tỷ đồng và một căn 100 m² được đoán 8.238 tỷ đồng.



Hình 5: Đường hồi quy tìm được vẽ đè lên 60 căn hộ trong dữ liệu, dấu sao đánh dấu dự đoán 6.671 tỷ đồng cho căn 80 m².

Hãy dành một phút nhìn Hình 5. Đường thẳng đi xuyên qua giữa đám điểm, có điểm nằm trên và có điểm nằm dưới, khá cân nhau ở cả hai phía. Đó chính là điều công thức đã hứa, vì đường thẳng buộc phải đi qua điểm trung bình của dữ liệu. Sinh viên cũng thấy các điểm bám khá sát đường, không có điểm nào lệch xa tới mức bất thường. Dấu đánh dấu trên đường là dự đoán cho căn 80 m², và nó nằm giữa vùng có nhiều dữ liệu nên ta khá yên tâm với con số 6.671 tỷ. Ở mục tiếp theo ta sẽ đo sự yên tâm đó bằng số chứ không chỉ bằng mắt.

Ghi nhớ

Với một biến đầu vào, w và b có công thức giải sẵn, không cần dò tìm. Tử số của w đo mức độ diện tích và giá đi cùng chiều, mẫu số đo độ phân tán của diện tích. Công thức $b = \bar{y} - w\bar{x}$ bảo đảm đường thẳng luôn đi qua điểm trung bình của dữ liệu. Trên bộ dữ liệu 60 căn hộ, cả cách tính tay lẫn scikit-learn đều cho $w = 0.078367$ và $b = 0.401752$. Sinh viên nên biết công thức để hiểu máy đang làm gì, và nên dùng thư viện để làm việc cho nhanh và ít sai.

6. Đánh giá mô hình

Ở mục trước chúng ta đã có một đường thẳng cụ thể. Câu hỏi tiếp theo là đường thẳng đó tốt tới mức nào. Chữ “tốt” phải được quy ra con số, vì nhìn bằng mắt thì ai cũng thấy đường nào cũng hợp lý. Mục này làm hai việc, chia lại dữ liệu cho công bằng rồi chấm điểm mô hình bằng bốn con số quen thuộc nhất trong học máy.

6.1. Vì sao không được chấm điểm trên chính dữ liệu đã học

Thầy kể một chuyện rất quen với sinh viên. Giả sử thầy cho lớp ôn đúng năm câu hỏi, rồi hôm thi đề ra đúng năm câu đó. Cả lớp được 10 điểm. Điểm 10 ấy không chứng minh được người

học hiểu bài, nó chỉ chứng minh họ nhớ được năm câu đã biết trước. Muốn biết sinh viên thực sự hiểu tới đâu, thầy phải ra một câu chưa ai từng ôn.

Mô hình cũng rơi vào đúng cái bẫy đó. Ở mục 5, hai số w và b được tính từ cả 60 căn hộ, rồi chúng ta lại đo sai số trên đúng 60 căn hộ ấy. Con số MSE bằng 0.1790 vì vậy là điểm của một bài thi trùng đề. Nó nói cho chúng ta biết đường thẳng bám sát 60 căn đã thấy tới đâu, nhưng không nói được gì về căn hộ thứ 61.

Hãy nhớ lại mục đích thật sự của công việc này. Chúng ta không xây mô hình để đoán giá 60 căn đã biết giá rồi. Chúng ta xây mô hình để đưa cho nó một căn hộ CHƯA từng thấy, và nó trả về một con số đáng tin. Muốn đo được khả năng đó thì phải giấu bớt một phần dữ liệu đi, không cho mô hình học, rồi lấy phần giấu đó ra làm đề thi.

Ghi nhớ

Học trên dữ liệu nào thì không được chấm điểm trên dữ liệu đó. Phần dữ liệu dùng để học gọi là tập học (*training set*). Phần dữ liệu giấu đi để chấm điểm gọi là tập kiểm tra (*test set*). Mô hình tuyệt đối không được nhìn thấy tập kiểm tra trong lúc học.

Hiện tượng vừa mô tả có tên riêng, và sinh viên sẽ nghe đi nghe lại cái tên này suốt cả môn học. Khi một mô hình thuộc lòng dữ liệu đã học nên điểm trên tập học rất cao, nhưng gặp dữ liệu mới thì sai nhiều, người ta nói mô hình bị quá khớp (*overfitting*). Đường thẳng của chúng ta quá đơn giản nên gần như không thể quá khớp, đó là một trong những lý do hồi quy tuyến tính hay được chọn làm mô hình đối chứng đầu tiên. Nhưng những mô hình phức tạp hơn ở các bài sau thì quá khớp rất dễ, và cách chia tập học với tập kiểm tra ở mục này chính là công cụ để phát hiện ra nó.

6.2. Chia tập học và tập kiểm tra

Thư viện `scikit-learn` có sẵn một hàm làm việc chia này, tên là `train_test_split`. Hàm nhận vào bảng đặc trưng và cột giá, rồi trả về bốn phần theo đúng thứ tự tập học của x , tập kiểm tra của x , tập học của y , tập kiểm tra của y . Sáu dòng dưới đây là toàn bộ phần chia dữ liệu.

```
15 # 80 phần trăm để học, 20 phần trăm để kiểm tra.
16 # random_state cố định để lần nào chia cũng ra đúng một cách chia.
17 X_train, X_test, y_train, y_test = train_test_split(
18     X, y, test_size=0.2, random_state=42
19 )
```

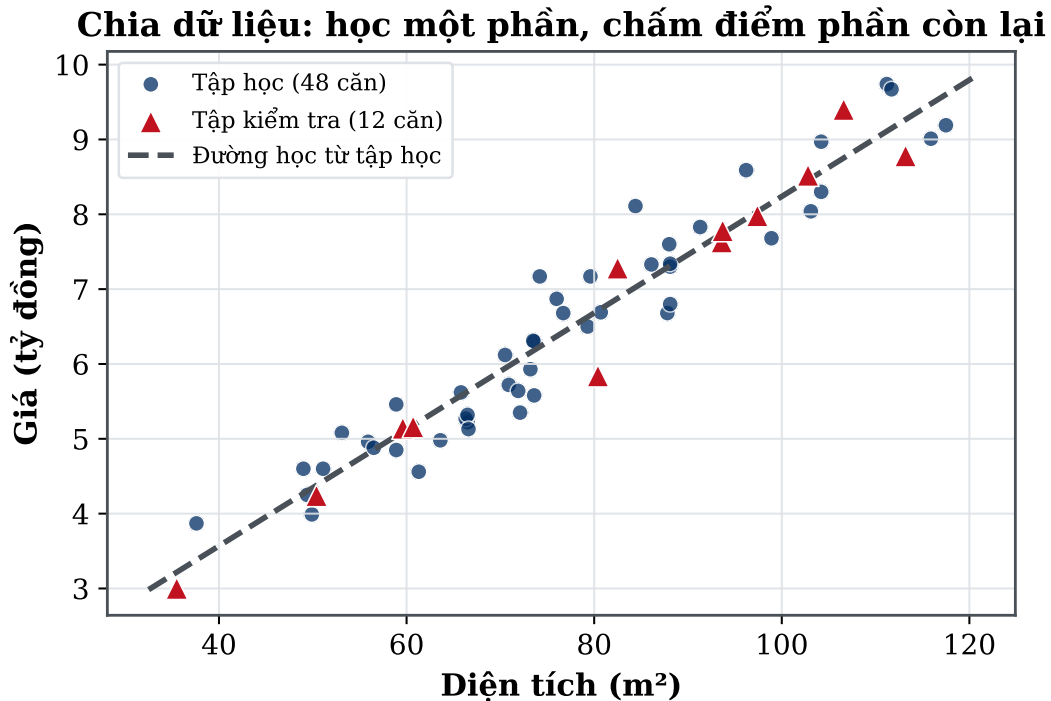
Tham số `test_size=0.2` nghĩa là giữ lại 20 phần trăm số căn hộ để kiểm tra. Chúng ta có 60 căn, 20 phần trăm của 60 là 12, nên tập kiểm tra có 12 căn và tập học còn 48 căn. Con số 0.2 là lựa chọn phổ biến khi dữ liệu nhỏ, vì giữ lại nhiều quá thì mô hình không còn đủ căn để học.

Tham số `random_state=42` mới là thứ hay bị hiểu nhầm. Hàm chia dữ liệu bốc ngẫu nhiên 12 căn, nên nếu không ràng buộc gì thì mỗi lần chạy sẽ ra một nhóm 12 căn khác nhau. Khi đó điểm số sẽ nhảy lung tung sau mỗi lần nhấn `Enter`, và sinh viên không biết mô hình thay đổi hay chỉ cách chia thay đổi. Đặt `random_state` một con số cố định là yêu cầu máy bốc ngẫu nhiên theo đúng một kiểu đã định trước. Nhờ vậy lần nào chạy cũng ra đúng 12 căn đó, kết quả in ra trong tài liệu này trùng khít với kết quả trên máy sinh viên.

Có một lý do thứ hai còn quan trọng hơn. Lát nữa chúng ta sẽ so sánh mô hình một biến với mô hình ba biến ở mục 8. Hai mô hình đó bắt buộc phải thi trên cùng một đề, nghĩa là cùng 12 căn kiểm tra. Nếu mỗi mô hình lại được chia dữ liệu một kiểu thì phần thắng có thể chỉ do bốc được đề dễ hơn.

Đừng thần thánh hóa số 42

Số 42 không có tính chất toán học đặc biệt nào cả, nó là một câu đùa quen thuộc trong giới lập trình. Sinh viên thay bằng 0, bằng 7 hay bằng 2026 đều được, kết quả sẽ hơi khác một chút và điều đó hoàn toàn bình thường. Điều bắt buộc là chọn xong thì giữ nguyên con số đó trong suốt cả bài thực hành.



Hình 6: Bốn mươi tám căn màu xanh dùng để học, mười hai căn hình tam giác đỏ được giấu đi để chấm điểm, đường nét đứt là đường học được từ riêng phần màu xanh.

Hình 6 cho sinh viên thấy rõ cách chia. Các tam giác đỏ nằm rải từ căn nhỏ nhất tới căn lớn nhất, không dồn về một phía, nên đề thi này không quá dễ cũng không quá khó. Đường nét đứt được vẽ chỉ từ 48 điểm xanh, máy chưa hề nhìn vào tam giác đỏ khi vẽ nó. Bây giờ việc chấm điểm chỉ còn là đo khoảng cách từ mỗi tam giác đỏ tới đường nét đứt.

6.3. Bốn con số để chấm điểm

Trước khi viết công thức, thầy quy ước ký hiệu cho gọn. Gọi n là số căn trong tập kiểm tra, ở đây $n = 12$. Gọi y_i là giá thật của căn thứ i , còn $\hat{y}_i$ là giá mô hình dự đoán cho căn đó. Dấu mũ trên đầu chữ y luôn có nghĩa là “do máy đoán ra”, còn chữ y trơn là số thật. Hiệu $y_i - \hat{y}_i$ chính là sai số của một căn, số dương nghĩa là mô hình đoán hụt, số âm nghĩa là mô hình đoán vống lên.

Độ đo thứ nhất là MAE (*mean absolute error*), nghĩa là sai số tuyệt đối trung bình. Ý tưởng đơn giản nhất có thể, lấy sai số của từng căn, bỏ dấu âm đi, rồi tính trung bình. Phải bỏ dấu âm vì nếu không thì một căn đoán hụt 0.3 và một căn đoán vống 0.3 sẽ triệt tiêu nhau thành 0, và mô hình tồi lại được khen là hoàn hảo.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Ưu điểm của MAE là đơn vị giữ nguyên là tỷ đồng, nên đọc ra là hiểu ngay trung bình mỗi căn lệch bao nhiêu tiền. Đây là con số thầy khuyên sinh viên đưa cho người không học máy xem.

Độ đo thứ hai là MSE (*mean squared error*), tên tiếng Việt là sai số toàn phương trung bình, đúng như đã gọi ở Mục 4. Thay vì bỏ dấu âm bằng trị tuyệt đối, MSE bỏ dấu âm bằng cách bình phương. Hệ quả là các cú trượt lớn bị phạt nặng hơn hẳn, vì trượt 0.8 đóng góp 0.64 trong khi trượt 0.2 chỉ đóng góp 0.04, gấp mười sáu lần dù khoảng cách chỉ gấp bốn.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Nhược điểm của MSE là đơn vị trở thành tỷ đồng bình phương, một thứ không ai hình dung nổi. Vì vậy MSE hợp để máy tối ưu hơn là để con người đọc, đúng như chúng ta đã dùng nó ở mục 5.

Độ đo thứ ba là RMSE (*root mean squared error*), chỉ đơn giản là căn bậc hai của MSE. Phép khai căn kéo đơn vị về lại tỷ đồng, nên RMSE vừa phạt nặng cú trượt lớn vừa đọc được bằng tiền.

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Độ đo thứ tư là R^2 , đọc là “R bình phương”, tiếng Việt hay gọi là hệ số xác định (*coefficient of determination*). Ba độ đo trên trả lời câu hỏi “lệch bao nhiêu tiền”, còn R^2 trả lời một câu khác, đó là “mô hình có khá hơn cách đoán ngu ngốc nhất không”. Cách đoán ngu ngốc nhất là không thèm nhìn diện tích, cứ căn nào cũng phán bằng giá trung bình $\bar{y}$, ở bộ dữ liệu này là 6.49 tỷ.

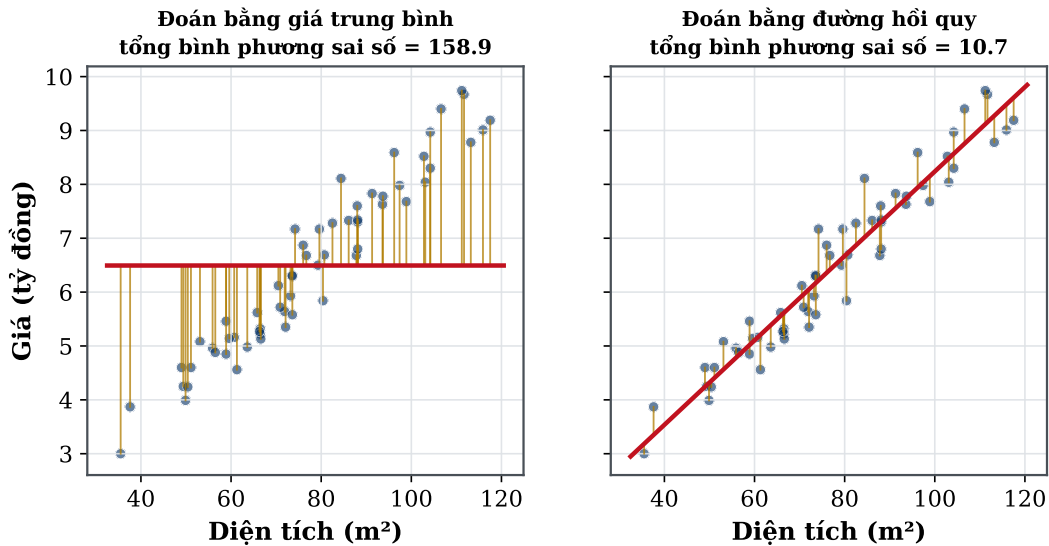
Công thức cần nhớ

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Tử số là tổng bình phương sai số của mô hình của chúng ta. Mẫu số là tổng bình phương sai số của mô hình ngây thơ luôn đoán bằng giá trung bình $\bar{y}$. Phân số nói mô hình của chúng ta còn sót lại bao nhiêu phần sai số so với cách đoán ngây thơ. Lấy 1 trừ đi phần sót lại thì ra phần biến động của giá mà mô hình giải thích được.

Có một chi tiết nhỏ về $\bar{y}$ mà sinh viên nên biết để khỏi thắc mắc về sau. Khi ta gọi `r2_score(y_test, y_pred)`, hàm này chỉ nhận được 12 căn của tập kiểm tra, nên $\bar{y}$ trong mẫu số là giá trung bình của đúng 12 căn đó chứ không phải 6.49 tỷ của cả 60 căn. Đây là quy ước chung của mọi thư viện, vì mô hình ngây thơ phải được chấm trên cùng bộ dữ liệu với mô hình thật thì so sánh mới công bằng.

Giá trị $R^2 = 1$ nghĩa là tử số bằng 0, mô hình đoán trúng phóc mọi căn. Giá trị $R^2 = 0$ nghĩa là tử số bằng mẫu số, mô hình không hơn gì việc phán bừa bằng giá trung bình. Giá trị âm là có thật và có nghĩa là mô hình còn tệ hơn cách đoán ngây thơ.



Hình 7: Cùng 60 căn hộ, bên trái đoán bằng giá trung bình nên các đoạn vàng rất dài, bên phải đoán bằng đường hồi quy nên các đoạn vàng ngắn hẳn lại.

Hình 7 cho sinh viên thấy R^2 ra từ đâu, bằng hai con số in ngay trên đầu mỗi bảng. Đoán bằng giá trung bình để lại tổng bình phương sai số là 158.9, đó chính là mẫu số trong công thức. Đoán bằng đường hồi quy để lại 10.7, đó là tử số. Lấy 10.7 chia 158.9 được khoảng 0.067, nghĩa là mô hình chỉ còn sót lại chừng 6.7 phần trăm sai số của cách đoán ngây thơ. Lấy 1 trừ 0.067 được khoảng 0.93, và đó là R^2 của đường hồi quy trên cả 60 căn. Con số 10.7 cũng có thể kiểm lại rất nhanh, vì MSE trên 60 căn là 0.1790 và 0.1790 nhân 60 đúng bằng 10.74.

Bảng 5: Bốn độ đo dùng để chấm điểm một mô hình hồi quy.

Độ đo	Công thức ngắn	Đơn vị	Tốt khi
MAE	trung bình của $ y_i - \hat{y}_i $	tỷ đồng	càng nhỏ càng tốt
MSE	trung bình của $(y_i - \hat{y}_i)^2$	tỷ đồng bình phương	càng nhỏ càng tốt
RMSE	$\sqrt{\text{MSE}}$	tỷ đồng	càng nhỏ càng tốt
R^2	$1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$	không có đơn vị	càng gần 1 càng tốt

6.4. Chấm điểm mô hình của chúng ta

Bây giờ ghép mọi thứ lại thành một chương trình hoàn chỉnh. Chương trình chia dữ liệu, huấn luyện trên 48 căn, dự đoán 12 căn còn lại, rồi in ra bốn con số. Điểm mẫu chốt nằm ở hai dòng gọi `fit` và `predict`, một dòng chỉ nhận `X_train` còn dòng kia chỉ nhận `X_test`.

```

1 # -*- coding: utf-8 -*-
2 """Bước 5: chia dữ liệu và chấm điểm mô hình bằng các độ đo chuẩn."""
3
4 import numpy as np
5 import pandas as pd
6 from sklearn.linear_model import LinearRegression
7 from sklearn.metrics import mean_absolute_error, mean_squared_error
8 from sklearn.metrics import r2_score
9 from sklearn.model_selection import train_test_split

```

```

10
11 df = pd.read_csv("data/gia_nha.csv")
12 X = df[["dien_tich"]]
13 y = df["gia"]
14
15 # 80 phần trăm để học, 20 phần trăm để kiểm tra.
16 # random_state cố định để lần nào chia cũng ra đúng một cách chia.
17 X_train, X_test, y_train, y_test = train_test_split(
18     X, y, test_size=0.2, random_state=42
19 )
20
21 print("Số can đo học      :", len(X_train))
22 print("Số can đo kiểm tra:", len(X_test))
23 print()
24
25 mo_hinh = LinearRegression()
26 mo_hinh.fit(X_train, y_train) # chỉ học trên tập học
27
28 y_pred = mo_hinh.predict(X_test) # chấm điểm trên tập kiểm tra
29
30 mae = mean_absolute_error(y_test, y_pred)
31 mse = mean_squared_error(y_test, y_pred)
32 rmse = np.sqrt(mse)
33 r2 = r2_score(y_test, y_pred)
34
35 print(f"MAE = {mae:.4f} tỷ đồng")
36 print(f"MSE = {mse:.4f}")
37 print(f"RMSE = {rmse:.4f} tỷ đồng")
38 print(f"R2 = {r2:.4f}")
39 print()
40
41 print("Một vài can trong tập kiểm tra:")
42 for dt, that, du_doan in list(zip(X_test["dien_tich"], y_test, y_pred))[:5]:
43     print(f" {dt:6.1f} m2   that = {that:5.2f}   du đoan = {du_doan:5.2f}"
44           f"   sai số = {that - du_doan:+5.2f}")

```

```

Số can đo học      : 48
Số can đo kiểm tra: 12

```

```

MAE = 0.2578 tỷ đồng
MSE = 0.1392
RMSE = 0.3730 tỷ đồng
R2 = 0.9622

```

```

Một vài can trong tập kiểm tra:

```

35.5 m2	that = 3.00	du đoan = 3.22	sai số = -0.22
50.4 m2	that = 4.24	du đoan = 4.38	sai số = -0.14
82.5 m2	that = 7.28	du đoan = 6.88	sai số = +0.40
93.6 m2	that = 7.63	du đoan = 7.74	sai số = -0.11

59.6 m2 that = 5.14 du doan = 5.09 sai so = +0.05

Thầy đọc từng dòng cho sinh viên. Hai dòng đầu xác nhận việc chia đúng như mong đợi, 48 căn để học và 12 căn để kiểm tra. Nói thêm một câu để sinh viên khỏi bối rối, mô hình trong mục này chỉ học từ 48 căn nên hệ số của nó là $w = 0.077837$ và $b = 0.454717$, hơi khác cặp số của Mục 5 vốn học từ cả 60 căn. Hai cặp số đều đúng, chúng chỉ đến từ hai lượng dữ liệu khác nhau, và chênh lệch nhỏ như vậy là dấu hiệu tốt cho thấy mô hình không quá nhạy với việc bớt đi vài căn. MAE bằng 0.2578 tỷ nghĩa là trung bình mỗi căn mô hình lệch khoảng 258 triệu đồng. Trên một căn hộ giá trung bình 6.49 tỷ thì mức lệch đó vào khoảng 4 phần trăm, nghe được với một mô hình chỉ nhìn duy nhất diện tích.

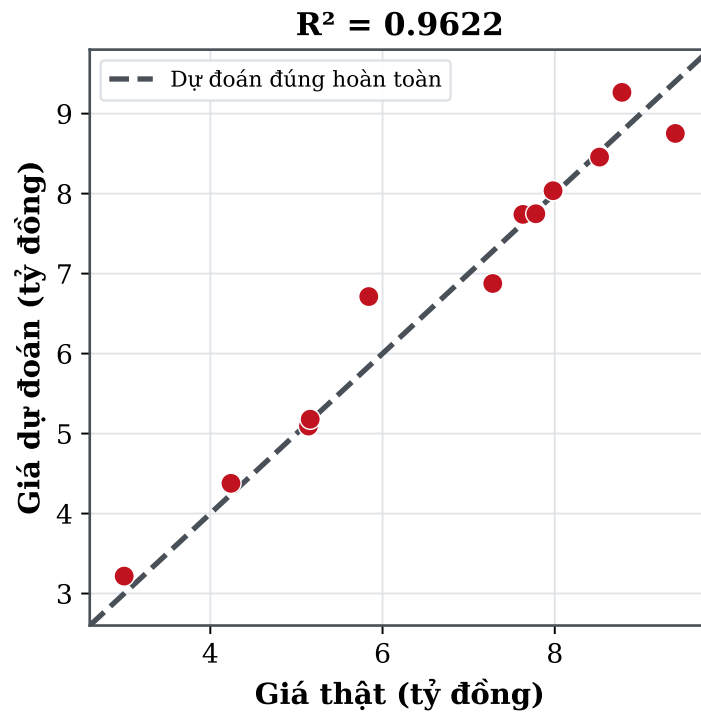
MSE bằng 0.1392 thì đừng có hiểu theo tiền, vì đơn vị của nó là tỷ đồng bình phương. RMSE bằng 0.3730 tỷ mới là con số đọc được, tức khoảng 373 triệu đồng. R^2 bằng 0.9622 nghĩa là mô hình giải thích được chừng 96 phần trăm biến động giá của 12 căn kiểm tra. Sinh viên để ý con số này cao hơn mức 0.93 tính trên cả 60 căn ở hình 7, và thầy sẽ nói ngay vì sao ở cuối mục.

Có một quy luật luôn đúng mà sinh viên nên nhớ. RMSE luôn lớn hơn hoặc bằng MAE, không bao giờ nhỏ hơn. Lý do là phép bình phương thổi phồng các sai số lớn lên trước khi lấy trung bình, nên cú trượt nặng kéo RMSE lên cao còn MAE thì không bị ảnh hưởng mạnh như vậy. Ở đây RMSE là 0.3730 còn MAE là 0.2578, chênh nhau khoảng 0.1152 tỷ. Khoảng chênh đó là một tín hiệu, nó cho biết trong 12 căn có vài căn bị trượt nặng hơn hẳn phần còn lại, chứ sai số không rải đều nhau. Nếu mọi căn đều lệch xấp xỉ như nhau thì hai con số này sẽ gần như bằng nhau.

Phần cuối của kết quả in ra năm căn cụ thể để sinh viên kiểm chứng bằng mắt. Căn 35.5 mét vuông giá thật 3.00 tỷ được đoán 3.22 tỷ, mô hình đoán vổng 0.22 tỷ. Căn 82.5 mét vuông giá thật 7.28 tỷ được đoán 6.88 tỷ, mô hình đoán hụt 0.40 tỷ. Căn 59.6 mét vuông thì gần như trúng, lệch vồn vện 0.05 tỷ. Dấu của cột lệch không giống nhau giữa các căn, đó là dấu hiệu tốt và mục sau sẽ giải thích vì sao.

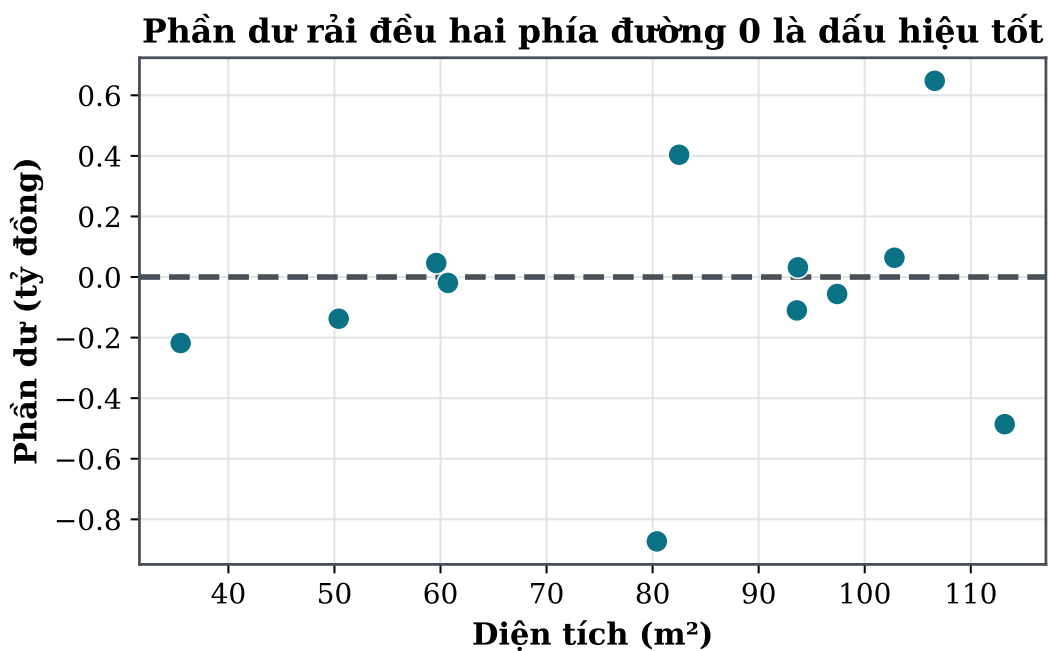
6.5. Nhìn sai số bằng hình

Bốn con số ở trên gói cả 12 căn vào bốn ô, tiện nhưng làm mất thông tin. Hai hình dưới đây mở gói đó ra.



Hình 8: Mỗi điểm là một căn trong tập kiểm tra, trục hoành là giá thật và trục tung là giá mô hình dự đoán.

Cách đọc hình 8 rất dễ. Đường chéo nét đứt là nơi giá dự đoán bằng đúng giá thật, nên một mô hình hoàn hảo sẽ có mọi điểm nằm đúng trên đường đó. Điểm nằm phía trên đường chéo là căn bị đoán vồng, điểm nằm phía dưới là căn bị đoán hụt. Khoảng cách theo chiều dọc từ một điểm tới đường chéo chính là sai số của căn đó. Ở hình này các điểm bám khá sát đường chéo và bám đều suốt từ căn rẻ nhất tới căn đắt nhất, nên mô hình không bị lệch riêng ở một khoảng giá nào.



Hình 9: Phần dư của 12 căn kiểm tra vẽ theo diện tích, đường ngang nét đứt là mức sai số bằng 0.

Hình 9 vẽ phần dư (*residual*), tức hiệu giữa giá thật và giá dự đoán của từng căn. Điểm nằm

trên đường 0 là căn mô hình đoán hụt, điểm nằm dưới là căn mô hình đoán vồng. Thứ cần tìm ở hình này không phải là điểm nào cao hay thấp, mà là hình dạng chung của đám điểm. Đám điểm cần rải đều hai phía đường 0 và không được theo một hình dạng nào cả, nghĩa là nhìn vào phải thấy lộn xộn. Lộn xộn ở đây là tin vui, vì nó cho biết phần mô hình chưa giải thích được chỉ còn là nhiễu ngẫu nhiên.

Ngược lại, có hai hình dạng báo động mà sinh viên phải biết nhận ra. Nếu các điểm xếp thành một đường cong, chẳng hạn âm ở hai đầu và dương ở giữa, thì quan hệ giữa diện tích và giá không thẳng, và một đường thẳng không bao giờ đủ. Nếu các điểm loe dần ra khi diện tích tăng, tức là căn càng to thì sai số càng lớn, thì mô hình đang kém tin cậy ở nhóm căn lớn dù bốn con số tổng kết vẫn đẹp. Gặp một trong hai hình dạng đó thì phải đổi mô hình chứ không phải đổi cách chấm điểm.

R^2 cao chưa chắc mô hình tốt

Con số 0.9622 trông rất đẹp nhưng nó được tính trên vòn vẹn 12 căn hộ. Với 12 điểm, chỉ cần bốc trúng hay bốc trượt vài căn khó là R^2 đã đổi khá nhiều. Bằng chứng nằm ngay trong bài, cũng đường hồi quy ấy nhưng chấm trên cả 60 căn thì R^2 chỉ khoảng 0.93. Còn một nửa lý do nữa, và nửa này ít người để ý. Mẫu số của R^2 là độ tản mát của giá trong chính nhóm căn đang chấm, nên nhóm nào giá trải rộng hơn thì mẫu số lớn hơn và R^2 dễ đẹp hơn. Mười hai căn kiểm tra của chúng ta có độ lệch chuẩn giá là 1.9194 tỷ, rộng hơn mức 1.6272 tỷ của cả 60 căn. Nói cách khác, bài kiểm tra này vô tình có nhiều căn dễ đoán ở hai đầu, nên điểm cao lên một phần là nhờ dễ chứ không hẳn nhờ mô hình giỏi. Sinh viên hãy thử đổi `random_state` sang vài giá trị khác và ghi lại R^2 từng lần, sẽ thấy con số nhảy. Vì vậy đừng bao giờ khoe một mình R^2 , hãy nói kèm số căn trong tập kiểm tra và kèm cả MAE. Trong thực tế người ta còn chia dữ liệu nhiều lần rồi lấy trung bình, kỹ thuật đó gọi là kiểm định chéo (*cross validation*) và sinh viên sẽ học ở bài sau.

7. Máy tự dò lời giải bằng gradient descent

Mục này giới thiệu cách máy tìm ra lời giải khi nó không được cho sẵn công thức. Tên của cách làm đó là xuống dốc theo độ dốc (*gradient descent*). Thầy báo trước để sinh viên yên tâm: mục này không có chứng minh toán học nào. Chúng ta chỉ cần hiểu máy đang làm gì ở mỗi vòng lặp, và nhìn con số sai số giảm dần qua từng vòng.

7.1. Vì sao vẫn cần cách này

Ở Mục 5 chúng ta đã có công thức tính thẳng ra hai số w và b . Thay số vào là xong, không phải thử, không phải đoán, và kết quả đúng ngay từ lần đầu. Vậy tại sao còn phải học thêm một cách khác chậm hơn?

Lý do là hồi quy tuyến tính một biến nằm trong một nhóm rất nhỏ các bài toán may mắn có công thức đóng (*closed-form solution*). Phần lớn mô hình học máy không có công thức như vậy. Mạng nơ ron (*neural network*) là ví dụ rõ nhất, vì một mạng cỡ nhỏ đã có hàng chục nghìn tham số và không ai giải tay ra được. Với những mô hình đó chỉ còn một cách, là dò dần từng bước nhỏ cho tới khi sai số không giảm nữa.

Thầy cố ý dạy cơ chế dò dần ngay trên bài toán giá nhà, dù bài này đã có công thức. Vì ta đã biết trước đáp án là $w = 0.078367$ và $b = 0.401752$, nên sau khi dò xong ta có thể đối chiếu. Nếu máy dò ra đúng hai số đó thì ta biết chắc mình đã cài đặt đúng. Học một kỹ thuật mới trên bài toán đã biết đáp án là cách học an toàn nhất. Sau này khi gặp bài không có đáp án, sinh viên sẽ dùng chính đoạn mã này với sự tự tin đã được kiểm chứng.

7.2. Ý tưởng người mù xuống núi

Hãy tưởng tượng sinh viên đang đứng trên sườn một quả đồi vào buổi sáng sương mù dày đặc. Sinh viên muốn xuống tới chỗ thấp nhất của thung lũng, nhưng nhìn ra xa chỉ thấy một màu trắng. Tuy vậy vẫn còn một thông tin dùng được, đó là cảm giác dưới bàn chân. Bàn chân cho biết phía nào là dốc lên và phía nào là dốc xuống, dù chỉ trong phạm vi một sải chân.

Vậy thì cứ bước một bước nhỏ về phía dốc xuống. Đứng lại, sờ chân lần nữa, rồi lại bước tiếp một bước nhỏ về phía dốc xuống. Lặp lại như thế đủ nhiều lần thì trước sau gì cũng tới chỗ bằng phẳng. Khi bàn chân thấy đất bằng, nghĩa là không còn hướng nào dốc xuống nữa, thì đó chính là đáy.

Bốn thứ trong câu chuyện này ứng với bốn thứ trong bài toán của chúng ta. Quả đồi chính là đường cong MSE ở hình 4 của Mục 4, đường cong hình chữ U có một đáy duy nhất. Vị trí người đó đang đứng chính là cặp giá trị w và b tại thời điểm hiện tại. Độ dốc dưới bàn chân chính là đạo hàm (derivative) của MSE theo w và theo b . Độ dài mỗi bước chân chính là tốc độ học (learning rate), một con số do chúng ta chọn.

Ghi nhớ

Máy không nhìn thấy toàn bộ đường cong sai số, giống như người trong sương mù không nhìn thấy thung lũng. Máy chỉ biết được độ dốc ngay tại chỗ nó đang đứng. Toàn bộ sức mạnh của gradient descent nằm ở chỗ chừng đó thông tin đã là đủ.

7.3. Ba dòng cốt lõi

Đoạn dưới đây là trái tim của cả mục, và nó chỉ dài hơn chục dòng.

```

26 for vong in range(1, so_vong + 1):
27     y_du_doan = w * x + b
28     # Chú ý: đây là dự đoán trừ giá thật, tức sai số của Mục 4 đảo dấu.
29     # Hai công thức độ dốc bên dưới viết theo đúng chiều này, đừng đổi dấu.
30     chenh_lech = y_du_doan - y
31
32     # Độ dốc của MSE theo w và theo b
33     grad_w = (2 / n) * (chenh_lech * x).sum()
34     grad_b = (2 / n) * chenh_lech.sum()
35
36     # Đi ngược hướng dốc thì sai số giảm
37     w -= toc_do_hoc * grad_w
38     b -= toc_do_hoc * grad_b

```

Mỗi vòng lặp làm đúng bốn việc theo thứ tự.

Việc thứ nhất là lấy w và b hiện tại để tính giá dự đoán cho cả 60 căn. Việc thứ hai là lấy dự đoán trừ đi giá thật. Xin sinh viên dừng lại ở đây một chút, vì đây đúng là sai số của Mục 4 nhưng đã bị đảo dấu. Mục 4 định nghĩa sai số là giá thật trừ giá đoán, còn ở đây ta lấy chiều ngược lại, nên mã đặt tên biến là `chenh_lech` chứ không gọi là sai số. Đảo dấu là có chủ ý, vì hai công thức độ dốc ngay dưới được viết theo đúng chiều đó. Nếu sinh viên tự sửa dòng này cho khớp định nghĩa Mục 4 mà không sửa gì thêm thì thuật toán sẽ leo dốc thay vì xuống dốc, và MSE tăng vọt sau vài vòng. Hai việc này chính là hai dòng `y_du_doan` và `chenh_lech`, nằm ngay đầu đoạn trích bên trên.

Việc thứ ba là tính độ dốc, tức hai dòng `grad_w` và `grad_b`. Hai dòng này trả lời câu hỏi nếu tăng w lên một chút thì MSE tăng hay giảm, và tăng giảm nhanh chừng nào. Đây là công

thức đạo hàm của MSE, đã được người ta tính sẵn từ lâu. Bài thực hành này không chứng minh chúng, sinh viên cứ dùng như dùng công thức diện tích hình tròn. Điều duy nhất cần nhớ là độ dốc dương nghĩa là đi tới sẽ leo lên, độ dốc âm nghĩa là đi tới sẽ tụt xuống.

Việc thứ tư là cập nhật, tức hai dòng `w -= toc_do_hoc * grad_w` và dòng tương tự cho `b`.

Quy tắc cập nhật

$$w \leftarrow w - \eta g_w \quad \text{và} \quad b \leftarrow b - \eta g_b$$

Mũi tên đọc là gán giá trị mới, giống dấu bằng trong Python. Ký hiệu g_w và g_b là độ dốc theo w và theo b , trong mã nguồn chúng tên là `grad_w` và `grad_b`. Ký hiệu η đọc là êta, đó là tốc độ học, trong mã nguồn nó tên là `toc_do_hoc`.

Điều quan trọng nhất trong quy tắc trên là dấu trừ. Độ dốc chỉ về hướng đi lên, mà chúng ta muốn đi xuống, nên phải đi ngược lại hướng đó. Nếu đổi dấu trừ thành dấu cộng thì máy sẽ leo ngược lên đỉnh đồi và sai số tăng vọt. Đây là lỗi gõ nhầm rất hay gặp, nên sinh viên hãy nhìn kỹ hai dấu trừ đó.

7.4. Một bước chuẩn bị không được bỏ

Trước vòng lặp có ba dòng lấy diện tích trừ đi trung bình rồi chia cho độ lệch chuẩn. Việc này gọi là chuẩn hoá (*standardization*), và nó không phải thủ tục trang trí.

Lý do rất cụ thể. Diện tích trong bảng là những con số hàng chục, trung bình 77.73 m² và độ lệch chuẩn khoảng 20 m². Độ dốc theo w được tính bằng sai số nhân với x rồi lấy trung bình, nên x càng lớn thì độ dốc càng lớn. Với x cỡ vài chục, độ dốc ở vòng đầu đã lên tới hàng trăm. Nhân độ dốc hàng trăm với tốc độ học 0.1 thì mỗi bước chân dài hàng chục, tức là nhảy vọt qua đáy sang tận sườn bên kia, và còn xa đáy hơn lúc xuất phát. Vòng sau lại nhảy xa hơn nữa, con số phình to rất nhanh, chỉ vài chục vòng là máy in ra `inf` hoặc `nan`. Sau khi chuẩn hoá thì gần như toàn bộ giá trị x nằm gọn trong khoảng từ -2 tới 2, độ dốc trở nên vừa phải, và thuật toán chạy êm.

Cái giá phải trả là w và b tìm được nằm trên thang đo đã chuẩn hoá, không phải thang mét vuông. Nhìn bảng kết quả ở phần sau sinh viên sẽ thấy w dừng ở 1.5713 và b dừng ở 6.4933, hai số này không giống đáp án của Mục 5. Chúng không sai, chúng chỉ đang nói bằng một đơn vị khác. Tiện thể đề ý con số $b = 6.4933$, đó đúng bằng giá trung bình của 60 căn, vì sau khi chuẩn hoá thì căn có diện tích trung bình ứng với $x = 0$. Hai dòng cuối của tệp mã làm việc đổi ngược lại, chia w cho độ lệch chuẩn và bù trừ phần trung bình vào b . Sau hai dòng đó ta mới có w và b đọc được theo mét vuông và tỷ đồng.

Đừng quên bước đổi ngược

Nếu chuẩn hoá mà quên đổi w và b về thang đo gốc, mô hình vẫn dự đoán ra số, nhưng số đó vô nghĩa. Dấu hiệu nhận ra là hệ số góc lớn bất thường so với đơn vị thực tế của dữ liệu.

7.5. Chạy thử và đối chiếu với đáp án

Dưới đây là toàn bộ tệp `b6_gradient_descent.py`.

```
1 # -*- coding: utf-8 -*-
2 """Bước 6: tự đi tìm w và b bằng cách dò từng bước nhỏ.
3
```

```

4 Đây là cách máy học khi bài toán không có công thức đóng. Với hồi quy tuyến
5 tính ta đã có công thức rồi, nên đoạn này chỉ để thấy cơ chế hoạt động.
6 """
7
8 import numpy as np
9 import pandas as pd
10
11 df = pd.read_csv("data/gia_nha.csv")
12 x_goc = df["dien_tich"].to_numpy()
13 y = df["gia"].to_numpy()
14
15 # Đưa diện tích về thang đo nhỏ quanh số 0. Thiếu bước này thuật toán sẽ vỡ.
16 x_tb = x_goc.mean()
17 x_do_lech = x_goc.std()
18 x = (x_goc - x_tb) / x_do_lech
19
20 w, b = 0.0, 0.0 # bắt đầu từ một đường thẳng nằm ngang đi qua gốc
21 toc_do_hoc = 0.1 # mỗi vòng lặp đi một bước dài bằng 0.1 lần độ dốc
22 so_vong = 200
23 n = len(x)
24
25 print("Vong      w      b      MSE")
26 for vong in range(1, so_vong + 1):
27     y_du_doan = w * x + b
28     # Chú ý: đây là dự đoán trừ giá thật, tức sai số của Mục 4 đảo dấu.
29     # Hai công thức độ dốc bên dưới viết theo đúng chiều này, đừng đổi dấu.
30     chenh_lech = y_du_doan - y
31
32     # Độ dốc của MSE theo w và theo b
33     grad_w = (2 / n) * (chenh_lech * x).sum()
34     grad_b = (2 / n) * chenh_lech.sum()
35
36     # Đi ngược hướng dốc thì sai số giảm
37     w -= toc_do_hoc * grad_w
38     b -= toc_do_hoc * grad_b
39
40     if vong in (1, 2, 5, 10, 25, 50, 100, 200):
41         # Tính Lại MSE bằng w và b VỪA cập nhật, để ba con số trên cùng một
42         # dòng bảng đều thuộc về cùng một thời điểm. Nếu dùng Lại chenh_lech
43         # ở trên thì MSE sẽ là của cặp w, b cũ và bảng bị lệch một vòng.
44         mse = ((w * x + b) - y) ** 2).mean()
45         print(f"vong:4d {w:7.4f} {b:7.4f} {mse:8.4f}")
46
47 print()
48 # Đổi w và b về lại thang đo mét vuông ban đầu
49 w_goc = w / x_do_lech
50 b_goc = b - w * x_tb / x_do_lech
51 print(f"Sau khi doi ve thang do met vuong:")
52 print(f" w = {w_goc:.6f}")

```

```

53 print(f"  b = {b_goc:.6f}")
54 print("So voi cong thuc o buoc 3: w = 0.078367, b = 0.401752")

```

Vong	w	b	MSE
1	0.3143	1.2987	28.7437
2	0.5657	2.3376	18.4604
5	1.0564	4.3656	4.9714
10	1.4025	5.7961	0.6936
25	1.5653	6.4688	0.1797
50	1.5712	6.4932	0.1790
100	1.5713	6.4933	0.1790
200	1.5713	6.4933	0.1790

Sau khi doi ve thang do met vuong:

w = 0.078367

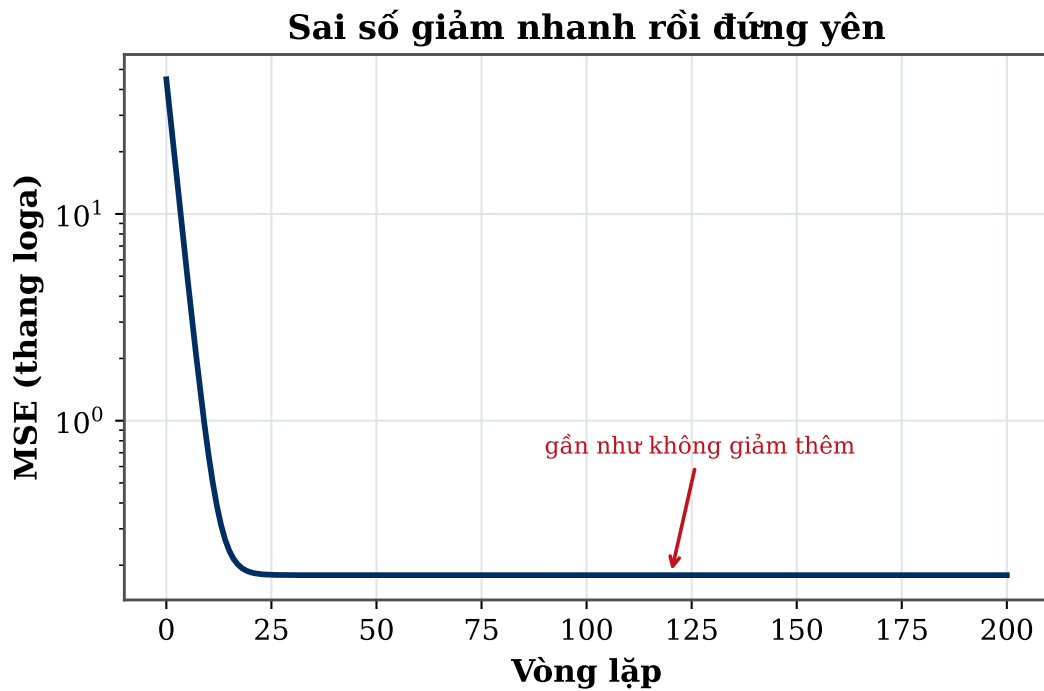
b = 0.401752

So voi cong thuc o buoc 3: w = 0.078367, b = 0.401752

Bảng này đáng đọc kỹ từng dòng vì nó kể lại hành trình xuống đồi. Trên mỗi dòng, ba con số w , b và MSE đều thuộc về cùng một thời điểm, tức là trạng thái sau khi đã chạy xong số vòng ghi ở cột đầu. Đường thẳng xuất phát nằm ngang đi qua gốc toạ độ, và MSE của nó là 44.8113, con số này chưa có trong bảng vì bảng bắt đầu từ vòng 1. Sau vòng 1, sai số đã rơi xuống 28.7437, tức là mất gần một phần ba chỉ sau một bước. Tới vòng 5 còn 4.9714, tới vòng 10 còn 0.6936. Tới vòng 25 còn 0.1797, và từ vòng 50 trở đi con số đứng yên ở 0.1790 suốt tới vòng 200.

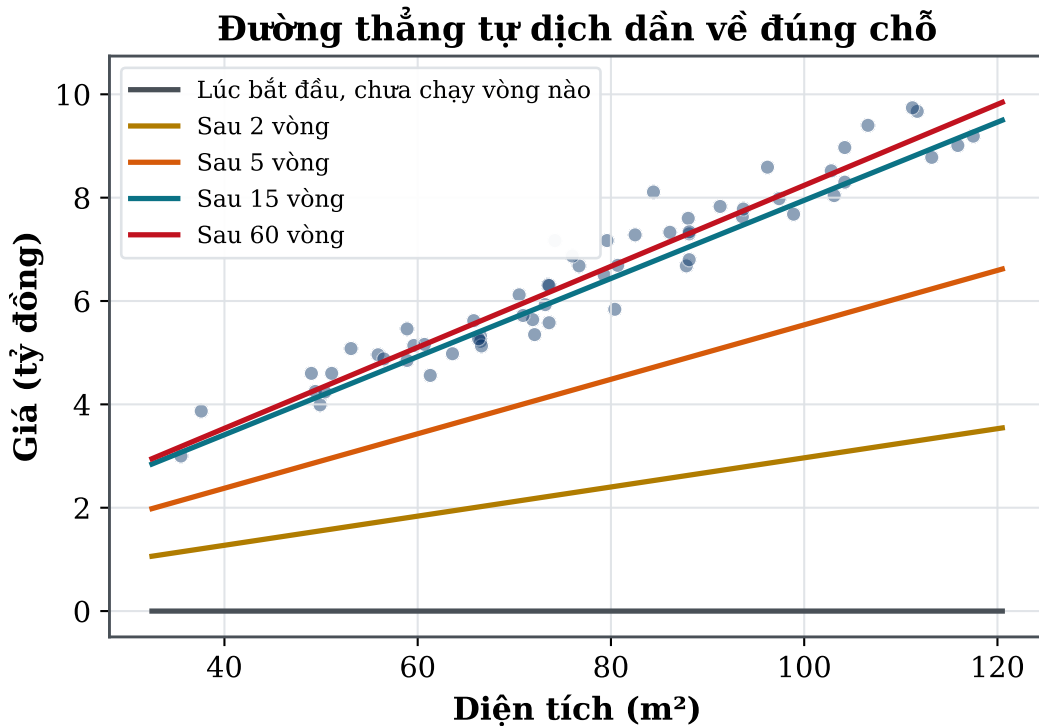
Đó là hình ảnh kinh điển của gradient descent. Lúc còn xa đáy thì sườn dốc, độ dốc lớn, nên mỗi bước đi được một quãng dài và sai số rơi rất nhanh. Càng gần đáy thì sườn càng thoải, độ dốc càng nhỏ, nên bước chân tự động ngắn lại và sai số gần như đứng yên. Vì vậy chạy thêm 150 vòng nữa cũng không cải thiện gì đáng kể.

Bây giờ tới phần quan trọng nhất của cả mục. Sau khi đổi về thang mét vuông, máy dò ra $w = 0.078367$ và $b = 0.401752$. Công thức ở Mục 5 cho $w = 0.078367$ và $b = 0.401752$. Hai kết quả trùng nhau tới chữ số thứ sáu sau dấu chấm, và MSE 0.1790 cũng trùng với MSE của nghiệm công thức. Sự trùng khít này không phải may mắn, nó là bằng chứng cho thấy hai cách làm đang giải cùng một bài toán. Công thức đi thẳng tới đáy trong một bước, gradient descent bò tới đáy qua nhiều bước nhỏ, nhưng cái đáy thì chỉ có một.



Hình 10: MSE giảm rất nhanh trong khoảng hai mươi vòng đầu rồi gần như nằm ngang, trục tung vẽ theo thang loga.

Trục tung của Hình 10 dùng thang loga (*logarithmic scale*), nghĩa là mỗi vạch không cộng thêm một lượng cố định mà nhân thêm một lần. Lý do phải làm vậy là sai số đi từ 44.8113 xuống 0.1790, tức là nhỏ đi khoảng 250 lần. Trên thang thường, toàn bộ đoạn từ vòng 25 trở đi sẽ bị ép dẹt xuống sát trục hoành và sinh viên không phân biệt nổi 0.1797 với 0.1790. Thang loga giãn phần số nhỏ ra, nhờ vậy một hình duy nhất cho thấy được cả giai đoạn sai số lớn lẫn giai đoạn sai số nhỏ.

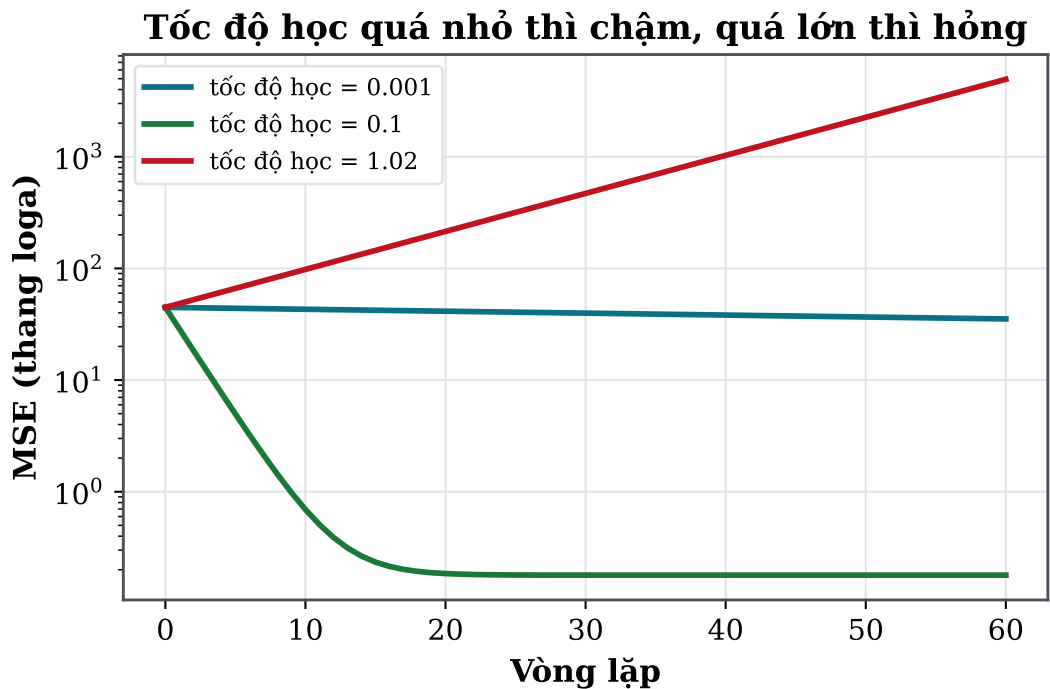


Hình 11: Đường thẳng của mô hình lúc bắt đầu và sau 2, 5, 15, 60 vòng lặp, mỗi vòng lại tiến gần vị trí đúng hơn một chút.

Hình 11 là cùng câu chuyện đó nhưng nhìn từ phía dữ liệu. Đường xám nằm bẹp trên trục hoành là lúc bắt đầu, khi w và b đều còn bằng 0 nên mô hình đoán mọi căn đều có giá 0. Các đường sau 2 và 5 vòng nhắc dần lên và nghiêng dần theo đám điểm. Đường sau 15 vòng đã gần như nằm đúng chỗ, và đường sau 60 vòng thì trùng với đường hồi quy ở Mục 5.

7.6. Tốc độ học, tham số phải tự chọn

Tốc độ học là độ dài bước chân, và chọn nó là việc của người dùng chứ không phải của máy. Hình dưới đây chạy lại đúng thuật toán trên với ba tốc độ học khác nhau, mỗi lần 60 vòng.



Hình 12: Ba tốc độ học trên cùng một bài toán, 0.001 quá chậm, 0.1 vừa đủ, 1.02 làm sai số tăng vọt.

Đường tốc độ học 0.001 đi xuống rất chậm và sau 60 vòng vẫn chưa tới đáy. Nó không sai, nó chỉ bước quá ngắn, giống như xuống đồi bằng những bước dài một xăng ti mét. Với tốc độ này ta phải chạy vài nghìn vòng mới tới nơi, nghĩa là tốn thời gian mà chẳng được gì thêm.

Đường tốc độ học 0.1 là đường chúng ta đã dùng trong cả mục này. Nó xuống nhanh trong vài chục vòng đầu rồi ổn định, và đó là đáng điệu ta mong muốn.

Đường tốc độ học 1.02 đi lên chứ không đi xuống. Bước chân dài quá nên từ sườn bên này nó nhảy vọt qua đáy sang sườn bên kia, và điểm mới còn cao hơn điểm cũ. Vòng sau nó lại nhảy ngược về, xa hơn nữa, nên sai số cứ thế tăng theo cấp số nhân. Điều đáng chú ý là 1.02 chỉ lớn hơn 1 một chút, vậy mà ranh giới giữa chạy được và hỏng hắc nằm ngay đó.

Tốc độ học không học được từ dữ liệu, vì nó là thứ ta phải đưa vào trước khi việc học bắt đầu. Những tham số kiểu này được gọi là siêu tham số (*hyperparameter*). Số vòng lặp cũng là một siêu tham số, và ở mục này thầy chọn 200. Cách chọn siêu tham số trong thực tế là thử vài giá trị rồi giữ lại giá trị cho kết quả tốt nhất. Nhưng phải nhớ đúng quy tắc của Mục 6, không được chấm điểm việc chọn đó trên tập kiểm tra. Chọn siêu tham số theo điểm của tập kiểm tra chính là để tập kiểm tra rò rỉ vào quá trình học, và điểm thu được sẽ lạc quan hơn thực tế. Người ta cắt thêm một phần dữ liệu thứ ba gọi là tập thẩm định (*validation set*) để làm việc chọn này, và sinh viên sẽ học cách làm ở bài sau.

Dấu hiệu chọn sai tốc độ học

Nếu MSE tăng dần qua các vòng thay vì giảm, hoặc màn hình hiện ra `nan` và `inf`, thì gần như chắc chắn tốc độ học đang quá lớn. Cách chữa đầu tiên luôn là giảm tốc độ học xuống mười lần, chẳng hạn từ 1.0 về 0.1, rồi chạy lại. Nếu vẫn hỏng thì giảm tiếp mười lần nữa. Còn nếu MSE giảm nhưng giảm quá chậm và hết số vòng vẫn chưa ổn định, thì làm ngược lại, tăng tốc độ học lên và chạy lại.

8. Dùng nhiều hơn một biến

Từ đầu bài tới giờ chúng ta chỉ cho máy nhìn đúng một con số của mỗi căn hộ, đó là diện tích. Mô hình một biến ấy đã chạy tốt, với $R^2 = 0.9622$ trên tập kiểm tra 12 căn. Nhưng khi đi xem nhà ngoài đời, không ai chỉ hỏi mỗi diện tích. Người mua còn hỏi căn đó có mấy phòng ngủ và nhà đã xây được bao nhiêu năm. Bộ dữ liệu `gia_nha.csv` có sẵn hai cột đó, là `so_phong` và `tuoi_nha`, mà nãy giờ chúng ta chưa đụng tới. Mục này đưa cả hai cột vào mô hình và xem kết quả khá lên được bao nhiêu.

8.1. Từ một đường thẳng sang một mặt phẳng

Giá một căn hộ đâu chỉ phụ thuộc diện tích. Hai căn cùng 80 m², một căn ba phòng ngủ và một căn hai phòng ngủ, thì giá thường không bằng nhau. Một căn mới bàn giao và một căn đã ở hai mươi năm cũng vậy. Mô hình một biến không có chỗ nào để chứa những thông tin đó, nên nó đành bỏ qua. Cách sửa rất tự nhiên: cho mỗi biến đầu vào một hệ số riêng, rồi cộng tất cả lại.

Công thức cần nhớ

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Ta đọc từng ký hiệu một cho rõ. $\hat{y}$ là giá mà mô hình dự đoán, đơn vị tỷ đồng. x_1 là diện tích tính bằng mét vuông, x_2 là số phòng ngủ, x_3 là tuổi nhà tính bằng năm. w_1, w_2, w_3 là ba hệ số góc, mỗi hệ số đi kèm đúng một biến và cho biết biến đó kéo giá lên hay dìm giá xuống. Còn b vẫn là hệ số chặn, đúng vai trò như ở mô hình một biến. Cách gọi tên trong sách là hồi quy tuyến tính bội (*multiple linear regression*), nhưng bản chất thì vẫn là phép cộng có trọng số mà thôi.

Về mặt hình học thì có một điểm đáng chú ý. Với một biến, mô hình là một đường thẳng nằm trên mặt phẳng, và chúng ta đã vẽ nó ra ở Mục 5. Với hai biến, mô hình trở thành một mặt phẳng trong không gian ba chiều, vẫn còn tưởng tượng được. Với ba biến trở lên thì không vẽ ra được nữa, vì chúng ta không nhìn thấy không gian bốn chiều. Điều may mắn là công thức và cách tính không đổi chút nào. Máy vẫn đi tìm bộ hệ số làm MSE nhỏ nhất, vẫn bằng đúng những phép toán cũ. Chuyện không vẽ được chỉ làm khó mắt người, chứ không làm khó máy.

8.2. Trong mã chỉ đổi đúng một chỗ

```
1 # -*- coding: utf-8 -*-
2 """Bước 7: dùng thêm số phòng và tuổi nhà, xem mô hình có khá hơn không."""
3
4 import pandas as pd
5 from sklearn.linear_model import LinearRegression
6 from sklearn.metrics import r2_score
7 from sklearn.model_selection import train_test_split
8
9 df = pd.read_csv("data/gia_nha.csv")
10 y = df["gia"]
11
12 # Chỉ đổi đúng một chỗ: danh sách cột đưa vào X
13 X_mot = df[["dien_tich"]]
14 X_ba = df[["dien_tich", "so_phong", "tuoi_nha"]]
15
16 for ten, X in [("Mot bien ", X_mot), ("Ba bien ", X_ba)]:
```

```

17     X_train, X_test, y_train, y_test = train_test_split(
18         X, y, test_size=0.2, random_state=42
19     )
20     mo_hinh = LinearRegression()
21     mo_hinh.fit(X_train, y_train)
22     r2 = r2_score(y_test, mo_hinh.predict(X_test))
23     print(f"{ten}: R2 tren tap kiem tra = {r2:.4f}")
24
25 print()
26
27 # Xem kỹ các hệ số của mô hình ba biến
28 X_train, X_test, y_train, y_test = train_test_split(
29     X_ba, y, test_size=0.2, random_state=42
30 )
31 mo_hinh = LinearRegression()
32 mo_hinh.fit(X_train, y_train)
33
34 print("He so cua tung bien:")
35 for ten_cot, he_so in zip(X_ba.columns, mo_hinh.coef_):
36     print(f"    {ten_cot:12s} {he_so:+.4f}")
37 print(f"    {'he so chan':12s} {mo_hinh.intercept_:+.4f}")

```

Sinh viên hãy so hai dòng tạo `X_mot` và `X_ba` trong đoạn mã trên. Dòng đầu lấy một cột, dòng sau lấy ba cột, và khác biệt nằm gọn trong danh sách tên cột đặt giữa hai dấu ngoặc vuông. Ngoài chỗ đó ra, không có dòng nào phải sửa. Vẫn là `LinearRegression()`, vẫn là `fit`, vẫn là `predict`, vẫn là `r2_score`. Đây chính là cái lợi của việc scikit-learn cho mọi mô hình dùng chung một giao diện (*interface*). Sinh viên học cách gọi một lần, rồi đổi dữ liệu hay đổi mô hình mà phần khung của chương trình vẫn giữ nguyên.

Chạy tệp `b7_nhieu_bien.py` thì màn hình in ra như sau.

```

Mot bien : R2 tren tap kiem tra = 0.9622
Ba bien  : R2 tren tap kiem tra = 0.9761

He so cua tung bien:
dien_tich    +0.0701
so_phong     +0.3042
tuoi_nha     -0.0295
he so chan   +0.6917

```

Kết quả cho thấy R^2 trên tập kiểm tra tăng từ 0.9622 lên 0.9761. Mức tăng này là có thật, và nó đi đúng hướng ta mong đợi khi cho máy biết thêm thông tin. Nhưng sinh viên đừng vội mừng, vì mức tăng đó không lớn. Phần biến động của giá mà mô hình chưa giải thích được chỉ giảm từ khoảng 3.8 phần trăm xuống khoảng 2.4 phần trăm, tính trên 12 căn của tập kiểm tra. Lý do là diện tích đã gánh gần hết công việc rồi. Ở bộ dữ liệu này, biết diện tích là gần như biết giá, nên hai cột thêm vào chỉ còn chỗ để tinh chỉnh.

8.3. Đọc hệ số của từng biến

Điều thú vị của hồi quy tuyến tính là ta đọc được từng hệ số (*coefficient*) và hiểu ngay nó nói gì. Bảng 6 chép lại ba hệ số vừa in ra và dịch từng con số sang lời nói thường ngày.

Bảng 6: Ba hệ số của mô hình ba biến và cách đọc chúng bằng lời.

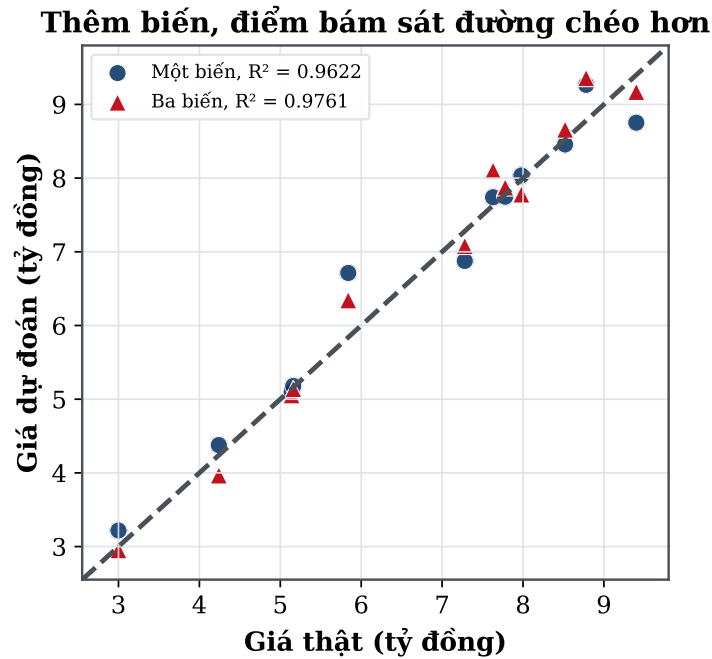
Biến	Hệ số	Đọc bằng lời
<code>dien_tich</code>	+0.0701	Thêm một mét vuông thì giá tăng khoảng 0.0701 tỷ, tức khoảng 70 triệu đồng
<code>so_phong</code>	+0.3042	Thêm một phòng ngủ thì giá tăng khoảng 0.3042 tỷ, tức khoảng 304 triệu đồng
<code>tuoi_nha</code>	-0.0295	Nhà cũ thêm một năm thì giá giảm khoảng 0.0295 tỷ, tức khoảng 30 triệu đồng

Mỗi dòng trong bảng đều ngầm kèm một điều kiện, đó là hai biến còn lại giữ nguyên. Câu đọc đúng của dòng đầu là: giữa hai căn cùng số phòng và cùng tuổi nhà, căn nào rộng hơn một mét vuông thì mô hình đoán đắt hơn khoảng 70 triệu đồng.

Bây giờ tới chỗ quan trọng nhất của mục này, xin sinh viên đọc kỹ. Hệ số của diện tích đã giảm, từ 0.0784 ở mô hình một biến xuống còn 0.0701 ở mô hình ba biến. Trước khi bàn nguyên nhân, sinh viên cần biết hai con số này chưa so thẳng với nhau được. Số 0.0784 tính trên cả 60 căn ở Mục 5, còn số 0.0701 chỉ tính trên 48 căn của tập học vì tệp `b7_nhieu_bien.py` chia dữ liệu trước rồi mới khớp mô hình. Nếu khớp cả hai mô hình trên đúng 48 căn đó thì hệ số diện tích đi từ 0.0778 xuống 0.0701, nên mức giảm vẫn còn nguyên. Cùng một cột dữ liệu, cùng một nhóm căn hộ, mà con số lại khác, nghe rất vô lý. Lý do nằm ở chỗ căn rộng thường có nhiều phòng ngủ hơn, hai cột này đi kèm nhau khá chặt trong bộ dữ liệu. Ở mô hình một biến, máy chỉ được nhìn thấy diện tích, nên nó dồn hết công lao vào hệ số diện tích. Hệ số 0.0784 khi đó gánh cả phần đóng góp thật của diện tích lẫn phần đóng góp của số phòng đi kèm. Khi ta đưa `so_phong` vào, cột này nhận lại phần công của mình, nên hệ số diện tích trả bớt và lùi về 0.0701. Không có con số nào sai cả, chỉ là hai con số trả lời hai câu hỏi khác nhau.

Cẩn thận

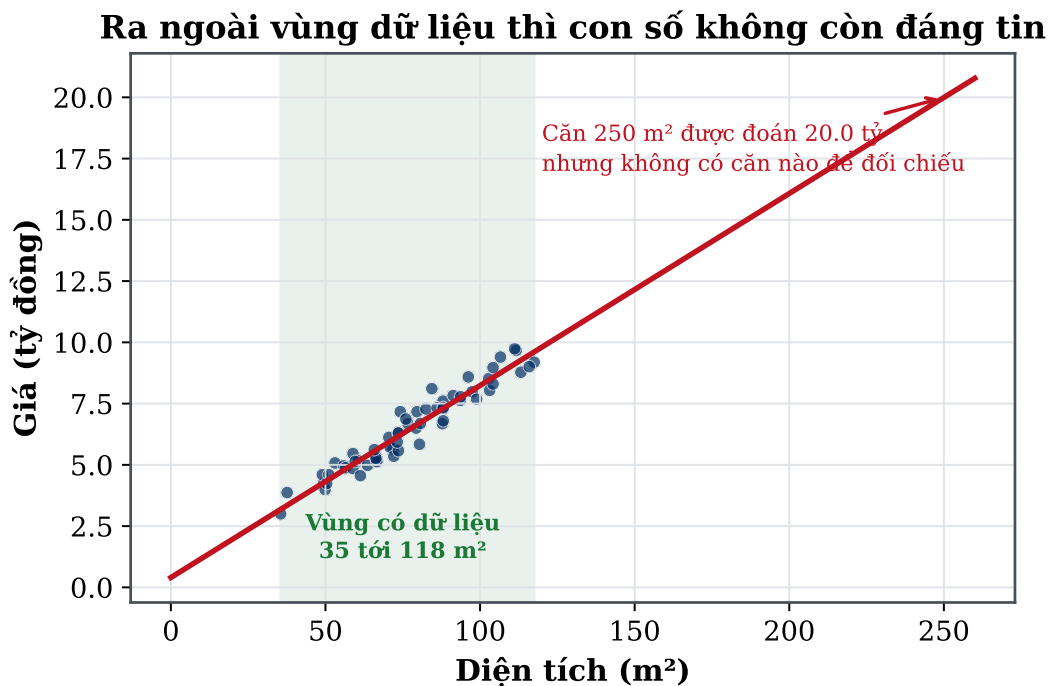
Không được đọc hệ số như một quan hệ nhân quả (*causation*). Hệ số chỉ mô tả quan hệ thống kê giữa các cột trong đúng bộ dữ liệu này. Nó không nói rằng cứ đập tường thêm một phòng ngủ là căn hộ của sinh viên có giá thêm 304 triệu đồng. Thêm nữa, khi hai cột có quan hệ chặt với nhau, hệ số của chúng trở nên khó diễn giải và dễ đổi giá trị mỗi khi ta thêm hoặc bớt biến. Đó đúng là chuyện vừa xảy ra với `dien_tich` và `so_phong` ở trên.



Hình 13: So sánh mô hình một biến và mô hình ba biến trên cùng 12 căn của tập kiểm tra, trục hoành là giá thật và trục tung là giá dự đoán.

Hình 13 đặt hai mô hình cạnh nhau trên cùng một đồ thị. Điểm tròn màu xanh là mô hình một biến, tam giác màu đỏ là mô hình ba biến, và đường chéo đứt nét là chỗ dự đoán trùng khớp giá thật. Điểm càng nằm gần đường chéo thì dự đoán càng chuẩn. Sinh viên sẽ thấy các tam giác đỏ bám đường chéo sát hơn một chút, đúng với con số R^2 nhỉnh hơn. Nhưng khác biệt giữa hai nhóm điểm nhỏ thôi, chứ không phải một bước nhảy vọt. Hình và con số nói cùng một câu chuyện, và đó là điều ta luôn mong muốn khi đọc kết quả.

8.4. Một giới hạn phải nhớ



Hình 14: Đường hồi quy kéo dài tới 250 m², vùng nền xanh nhạt là khoảng diện tích thật sự có dữ liệu.

Mô hình vừa xây chỉ đáng tin trong khoảng diện tích từ 35.5 tới 117.5 m². Đó là khoảng nhỏ nhất và lớn nhất trong 60 căn hộ của chúng ta, và trên Hình 14 nó được tô nền xanh nhạt. Trong vùng đó, mỗi lần mô hình đưa ra một con số thì đều có những căn hộ thật nằm quanh để đối chiếu. Việc dùng mô hình ở ngoài khoảng dữ liệu được gọi là ngoại suy (*extrapolation*). Đưa diện tích 250 m² vào công thức thì máy vẫn vui vẻ trả về khoảng 20.0 tỷ đồng, không báo lỗi gì. Nhưng trong 60 căn của ta, căn lớn nhất mới 117.5 m², nghĩa là không có một căn nào để kiểm chứng con số 20.0 tỷ đó. Máy không hề học được gì về biệt thự, nó chỉ kéo dài cái đường thẳng đã học ra xa thêm. Ngoài đời, nhà 250 m² thường là một loại bất động sản khác hẳn, với cách định giá khác hẳn. Vì vậy khi trình bày kết quả, sinh viên nên nói rõ mô hình dùng được trong khoảng nào. Một mô hình biết nói câu “ngoài khoảng này tôi không chắc” thì đáng tin hơn nhiều so với một mô hình trả lời mọi thứ.

Ghi nhớ

Thêm biến vào mô hình chỉ là thêm tên cột vào danh sách trong ngoặc vuông, phần còn lại của mã giữ nguyên. Hệ số của một biến có thể đôi khi ta thêm biến khác, và điều đó bình thường chứ không phải lỗi. Mô hình chỉ đáng tin trong khoảng dữ liệu mà nó đã được học. Mục tiếp theo là phần bài tập. Sinh viên hãy tạo một thư mục riêng cho bài làm của mình, chép dữ liệu vào đó rồi mới sửa mã, để các tệp mẫu của bài thực hành luôn còn nguyên khi cần đối chiếu.

9. Bài tập

Từ đây trở đi sinh viên làm một mình, không chép lại kết quả của tài liệu. Hãy tạo một thư mục tên `baitap01` nằm ngay trong thư mục bài thực hành, cạnh thư mục `data` và thư mục `code`. Mỗi bài viết thành một tệp Python riêng và đặt tên theo số bài, ví dụ `bai1.py` cho bài thứ nhất. Thầy bắt tách tệp vì một lý do rất thực tế, nếu một bài chạy lỗi thì năm bài còn lại vẫn chấm được bình thường. Cả sáu bài đều dùng đúng một bộ dữ liệu `data/gia_nha.csv` với 60 căn hộ mà chúng ta đã làm việc suốt buổi hôm nay. Sinh viên không phải đi tìm dữ liệu mới, cũng không được tự sửa dữ liệu. Có một chi tiết nhỏ hay làm sinh viên mất thời gian, đó là đường dẫn tới tệp dữ liệu. Quy tắc của cả bài thực hành vẫn giữ nguyên, sinh viên luôn chạy lệnh từ thư mục gốc của bài chứ không `cd` vào `baitap01`. Nghĩa là gõ `python baitap01/bai1.py`, và trong mã vẫn viết `data/gia_nha.csv` y hệt các tệp mẫu ở những mục trước. Lý do là Python tính đường dẫn tương đối theo thư mục mà sinh viên đang đứng lúc gõ lệnh, chứ không theo chỗ tệp mã nằm. Nhờ giữ đúng một quy tắc này mà sinh viên chép tệp mẫu sang `baitap01` là chạy được ngay, không phải sửa dòng `pd.read_csv`. Bài nộp gồm hai thứ, tệp mã nguồn và ảnh chụp màn hình (*screenshot*) phần kết quả in ra. Ảnh chụp phải nhìn thấy được cả dòng lệnh sinh viên gõ lẫn những gì chương trình in ra, vì thầy cần biết con số đó đến từ lần chạy thật.

Bài tập 1: Lọc ra nhóm căn hộ lớn

Đọc bộ dữ liệu, sau đó in ra hai con số. Con số thứ nhất là số căn có diện tích lớn hơn 100 mét vuông. Con số thứ hai là giá trung bình của riêng nhóm căn đó, tính bằng tỷ đồng. Bài này chỉ cần các hàm đã dùng ở mục 3, đó là `pd.read_csv` để đọc tệp và `mean` để lấy trung bình. Gợi ý, pandas cho phép lọc bằng một điều kiện đúng sai (*boolean mask*), sinh viên viết `nhom_lon = df[df["dien_tich"] > 100]` rồi đếm bằng `len(nhom_lon)`. Giá trung bình của nhóm lấy bằng `nhom_lon["gia"].mean()`. Toàn bộ 60 căn có diện tích trải từ 35.5 tới 117.5 mét vuông, nên nhóm sinh viên lọc ra sẽ nhỏ hơn 60 khá nhiều. Nộp tệp

`bai1.py` và ảnh chụp kết quả.

Bài tập 2: Vẽ biểu đồ phân tán theo số phòng

Vẽ biểu đồ phân tán với trục hoành là cột `so_phong` và trục tung là cột `gia`. Đặt tên cho hai trục và đặt tiêu đề cho hình, vì một hình không có nhãn thì người xem không hiểu. Lưu hình thành tệp `bai2.png` rồi viết một câu nhận xét về xu hướng sinh viên nhìn thấy.

Bài này dùng `matplotlib.pyplot` giống hệt cách chúng ta đã vẽ hình đầu tiên của bài thực hành, chỉ đổi cột đưa vào `scatter`. Gợi ý, hãy gọi `plt.savefig("bai2.png", dpi=150)` trước khi gọi `plt.show()`, vì lệnh `show` đóng hình lại và sau đó lưu ra sẽ được một tệp trắng. Câu nhận xét viết bằng tiếng Việt, đặt trong lệnh `print` để nó hiện ra trong ảnh chụp. Nộp tệp `bai2.py`, tệp hình `bai2.png` và ảnh chụp kết quả.

Bài tập 3: Đổi biến đầu vào sang tuổi nhà

Dùng lại đúng công thức bình phương tối thiểu ở mục 5, nhưng đổi biến đầu vào từ `dien_tich` sang `tuoi_nha`. Biến cần dự đoán vẫn là `gia`. In ra hệ số góc w và hệ số chặn b với sáu chữ số sau dấu phẩy. Sau đó nhận xét dấu của w và giải thích bằng lời vì sao nó mang dấu âm.

Cách nhanh nhất là chép tệp `b3_cong_thuc.py` sang thư mục `baitap01` rồi đổi đúng một dòng, dòng chọn cột cho biến x . Gợi ý cho phần giải thích, w là mức thay đổi của giá khi tuổi nhà tăng thêm một năm. Khi chạy với `dien_tich` chúng ta được $w = 0.078367$ và $b = 0.401752$, con số của sinh viên lần này sẽ khác hẳn nên đừng hoảng.

Có một câu hỏi thêm mà sinh viên nào làm xong sớm nên thử trả lời. Hãy vẽ biểu đồ phân tán giữa `tuoi_nha` và `gia` rồi nhìn kỹ. Dấu âm của w là đúng, nhưng các điểm có thật sự bám quanh một đường thẳng như ở biểu đồ diện tích không? Nếu không thì con số w này đáng tin tới mức nào, và một hệ số có dấu đúng đã đủ để kết luận hai đại lượng có quan hệ chặt chưa? Nộp tệp `bai3.py`, ảnh chụp kết quả và câu giải thích.

Bài tập 4: Thêm số phòng vào mô hình

Huấn luyện một mô hình hồi quy tuyến tính bằng `scikit-learn` với hai cột đầu vào là `dien_tich` và `so_phong`. Chia dữ liệu thành tập học và tập kiểm tra rồi in ra R^2 trên tập kiểm tra. So sánh con số của sinh viên với $R^2 = 0.9622$ của mô hình một biến trong mục 6 và nói xem thêm cột này có đáng hay không.

Bài này dùng `train_test_split`, `LinearRegression` và `r2_score`, cả ba đều đã xuất hiện ở mục 6 và mục 8. Hãy giữ nguyên `test_size=0.2` và `random_state=42` như trong tài liệu. Đó là điều kiện để sinh viên vẫn chia được 48 căn để học và 12 căn để kiểm tra đúng như tài liệu, nhờ vậy phép so sánh mới công bằng. Gợi ý, so với tệp `b7_nhieu_bien.py` sinh viên chỉ cần sửa danh sách cột bên trong hai cặp ngoặc vuông của `X`. Nộp tệp `bai4.py`, ảnh chụp kết quả và câu so sánh.

Bài tập 5: Thử hai tốc độ học khác

Chép tệp `b6_gradient_descent.py` sang thư mục `baitap01` rồi chạy lại hai lần. Lần thứ nhất đặt tốc độ học bằng 0.001, lần thứ hai đặt bằng 1.02. Mỗi lần hãy ghi lại giá trị MSE ở vòng lặp thứ 200, rồi giải thích vì sao hai con số đó khác nhau.

Trong tài liệu chúng ta chạy với tốc độ học 0.1 và sau 200 vòng thì MSE dừng ở 0.1790. Hãy giữ nguyên `so_vong = 200` và chỉ sửa đúng dòng gán `toc_do_hoc`, vì đôi hai thứ cùng lúc thì không biết cái nào gây ra khác biệt. Gợi ý, hãy nhớ lại phần so sánh ba tốc độ học ở mục 7, một bước quá ngắn thì đi chưa tới đáy còn một bước quá dài thì nhảy vọt qua đáy rồi ra xa hơn. Với tốc độ học 1.02, con số MSE sẽ lớn tới mức kỳ quặc, và đó là kết quả đúng chứ không phải máy hỏng.

Làm thêm nếu còn thời gian, hãy thử tốc độ học 0.5 rồi 0.9. Sinh viên sẽ thấy cả hai vẫn về đúng 0.1790, nghĩa là ranh giới giữa chạy được và hỏng không nằm ở chỗ ta tưởng. Hãy dò xem ranh giới đó nằm quanh con số nào. Nộp tệp `bai5.py`, hai ảnh chụp kết quả và phần giải thích.

Bài tập 6: Viết hàm dự đoán có cảnh báo

Viết một hàm tên `du_doan_gia(dien_tich)` nhận vào diện tích và trả về giá dự đoán theo mô hình một biến. Bên trong hàm, nếu diện tích nằm ngoài khoảng từ 35.5 tới 117.5 mét vuông thì in thêm một dòng cảnh báo trước khi trả về kết quả. Gọi thử hàm với ba giá trị là 60, 80 và 200 mét vuông rồi in cả ba kết quả ra màn hình.

Sinh viên không phải huấn luyện lại gì cả, chỉ cần dùng lại hai con số đã có là $w = 0.078367$ và $b = 0.401752$. Gợi ý, hãy khai báo hàm bằng `def`, trả kết quả bằng `return`, và viết cảnh báo trong một câu lệnh `if` với điều kiện `dien_tich < 35.5 or dien_tich > 117.5`. Có một mốc để sinh viên tự kiểm tra, gọi hàm với 80 mét vuông phải ra 6.671 tỷ đồng, đúng bằng con số ở mục 5. Lý do phải cảnh báo là căn 200 mét vuông nằm xa mọi căn trong dữ liệu, mô hình chưa từng thấy vùng đó nên con số nó đưa ra chỉ là suy đoán kéo dài. Nộp tệp `bai6.py` và ảnh chụp ba kết quả.

Tiêu chí chấm

Bốn tiêu chí dưới đây áp dụng cho cả sáu bài, mỗi tiêu chí chiếm một phần tư số điểm của bài đó.

- Mã chạy được ngay, không báo lỗi và không cần người chấm sửa đường dẫn.
- Kết quả in ra đúng và khớp với ảnh chụp màn hình sinh viên nộp.
- Có nhận xét bằng lời ở những bài yêu cầu, viết thành câu hoàn chỉnh chứ không phải một từ.
- Tên biến đặt có nghĩa, ví dụ `nhom_lon` hoặc `gia_du_doan`, tránh đặt tên kiểu `a` và `b1`.

10. Phụ lục A, bảng tra nhanh

Phần này gom lại những thứ sinh viên cần tra lại nhiều lần khi làm bài. Sinh viên không cần thuộc lòng các bảng dưới đây. Người đi làm thật cũng mở tài liệu ra tra suốt, không ai nhớ hết cú pháp. Điều cần nhớ là mỗi lệnh dùng để làm gì, còn cách gõ chính xác thì lật lại phụ lục này.

10.1. Các hàm đã dùng trong bài

Bảng 7 liệt kê các lệnh sinh viên cần cho bài thực hành này. Cột giữa cho biết lệnh thuộc thư viện nào, vì gõ đúng tên mà quên dòng `import` thì Python vẫn báo lỗi. Ba lệnh vẽ hình `plt.scatter`, `plt.plot` và `plt.savefig` không xuất hiện trong bảy tệp mã, chúng được liệt kê ở đây vì Bài tập 2 cần tới. Cả ba nằm trong thư viện `matplotlib`, thường được gọi qua tên rút gọn `plt`.

Bảng 7: Các hàm và thuộc tính đã dùng trong bài thực hành, kèm thư viện chứa chúng.

Lệnh	Thuộc thư viện	Công dụng
<code>pd.read_csv</code>	pandas	Đọc một tệp csv trên đĩa thành bảng dữ liệu.
<code>df.shape</code>	pandas	Cho biết bảng có bao nhiêu dòng và bao nhiêu cột.
<code>df.head</code>	pandas	In ra vài dòng đầu của bảng để nhìn thử.
<code>df.describe</code>	pandas	Tính nhanh số lượng, trung bình, độ lệch chuẩn, nhỏ nhất và lớn nhất của từng cột số.
<code>df.isna</code>	pandas	Đánh dấu các ô bị bỏ trống, thường gọi kèm <code>.sum()</code> để đếm.
<code>df[["dien_tich"]]</code>	pandas	Lấy một cột nhưng giữ kết quả ở dạng bảng hai chiều.
<code>.to_numpy</code>	pandas	Đổi bảng hoặc cột sang mảng số của numpy để tính toán.
<code>.mean</code>	pandas, numpy	Tính giá trị trung bình.
<code>.sum</code>	pandas, numpy	Tính tổng các phần tử.
<code>np.sqrt</code>	numpy	Lấy căn bậc hai, dùng để đổi MSE thành RMSE.
<code>plt.scatter</code>	matplotlib	Vẽ các điểm rời rạc, mỗi căn nhà là một chấm.
<code>plt.plot</code>	matplotlib	Vẽ đường nối, dùng để vẽ đường hồi quy.
<code>plt.savefig</code>	matplotlib	Lưu hình đang vẽ ra một tệp ảnh.
<code>train_test_split</code>	scikit-learn	Chia dữ liệu thành tập học và tập kiểm tra.
<code>LinearRegression</code>	scikit-learn	Tạo một mô hình hồi quy tuyến tính chưa được huấn luyện.
<code>.fit</code>	scikit-learn	Huấn luyện mô hình, tức là tìm w và b từ dữ liệu.
<code>.predict</code>	scikit-learn	Dùng mô hình đã huấn luyện để đoán giá cho dữ liệu mới.
<code>.coef_</code>	scikit-learn	Đọc hệ số góc w mà mô hình đã học được.
<code>.intercept_</code>	scikit-learn	Đọc hệ số chặn b mà mô hình đã học được.
<code>mean_absolute_error</code>	scikit-learn	Tính MAE, trung bình khoảng cách giữa giá đoán và giá thật.
<code>mean_squared_error</code>	scikit-learn	Tính MSE, tức sai số toàn phương trung bình.
<code>r2_score</code>	scikit-learn	Tính R^2 , mức độ mô hình giải thích được biến thiên của giá.

10.2. Các ký hiệu toán dùng trong bài

Toàn bài chỉ dùng mười ba ký hiệu, và sinh viên đã gặp đủ cả mười ba. Bảng 8 gom chúng lại một chỗ. Khi gặp một công thức trông đáng sợ, sinh viên hãy đọc từng ký hiệu theo bảng này trước, rồi mới đọc cả công thức.

Bảng 8: Ý nghĩa của các ký hiệu toán xuất hiện trong bài thực hành.

Ký hiệu	Ý nghĩa
x	Giá trị đầu vào, trong bài này là diện tích căn hộ tính bằng mét vuông.
y	Giá thật của căn hộ, tính bằng tỷ đồng, đây là con số chúng ta muốn đoán.
$\hat{y}$	Giá mà mô hình đoán ra, đọc là y mũ, luôn khác y một chút.
w	Hệ số góc, cho biết giá tăng bao nhiêu tỷ khi diện tích tăng thêm một mét vuông.
b	Hệ số chặn, là giá trị của đường thẳng khi x bằng 0.
n	Số căn hộ trong tập dữ liệu đang xét.
i	Chỉ số đánh số từng căn, chạy từ 1 tới n , nên x_i là diện tích của căn thứ i .
$\bar{x}$	Trung bình của tất cả các x , đọc là x gạch ngang.
$\bar{y}$	Trung bình của tất cả các y , đọc là y gạch ngang.
MSE	Sai số toàn phương trung bình, con số càng nhỏ thì đường thẳng càng khớp dữ liệu.
RMSE	Căn bậc hai của MSE, có cùng đơn vị với giá nên dễ hình dung hơn MSE.
MAE	Trung bình trị tuyệt đối sai số, cho biết trung bình mô hình đoán lệch bao nhiêu tỷ.
R^2	Phân biến thiên của giá mà mô hình giải thích được, bằng 1 là hoàn hảo, bằng 0 là dở như đoán bằng trung bình.

10.3. Các con số chính của bài

Bảng 9 là toàn bộ kết quả số của bài, chép lại từ đúng những gì chương trình in ra. Khi sinh viên chạy lại bảy tệp mã, các con số này phải trùng tới từng chữ số. Nếu có con số nào lệch, gần như luôn là do một dòng mã bị sửa nhầm hoặc do quên đặt `random_state=42` khi chia dữ liệu.

Bảng 9: Các con số kết quả của bài thực hành, đọc trực tiếp từ kết quả chạy trong thư mục outputs.

Đại lượng	Giá trị
Hệ số góc w của mô hình một biến	0.078367
Hệ số chặn b của mô hình một biến	0.401752
MSE trên toàn bộ 60 căn	0.1790
Dự đoán cho căn 80 m ²	6.671 tỷ đồng
Dự đoán cho căn 100 m ²	8.238 tỷ đồng
Số căn trong tập học	48
Số căn trong tập kiểm tra	12
MAE trên tập kiểm tra	0.2578 tỷ đồng
MSE trên tập kiểm tra	0.1392
RMSE trên tập kiểm tra	0.3730 tỷ đồng
R^2 trên tập kiểm tra, mô hình một biến	0.9622
R^2 trên tập kiểm tra, mô hình ba biến	0.9761

Ba con số đáng nhớ nhất

Nếu chỉ được nhớ ba con số của bài này thì hãy nhớ w bằng 0.078367, RMSE trên tập kiểm tra bằng 0.3730 tỷ đồng, và R^2 bằng 0.9622. Con số thứ nhất là điều mô hình đã học được. Con số thứ hai nói mô hình sai trung bình khoảng bao nhiêu tiền. Con số thứ ba nói mô hình tốt hơn việc đoán bừa bằng giá trung bình đến mức nào.

11. Phụ lục B, lỗi thường gặp

Gặp lỗi là chuyện bình thường khi lập trình, kể cả với người đã làm nhiều năm. Sinh viên mới thường hoảng khi thấy một khối chữ đỏ dài trên màn hình. Thật ra Python báo lỗi khá rõ ràng và hầu như luôn chỉ đúng dòng bị sai. Bảng 10 gom những lỗi mà sinh viên hay gặp nhất khi làm đúng bài thực hành này.

Trước khi tra bảng, sinh viên hãy kiểm tra một điều đơn giản đã. Mã nguồn phải được chạy từ thư mục gốc của bài, không phải từ bên trong thư mục code.

```
cd C:\MayHoc\bai01_hoi_quy
python code/b1_doc_du_lieu.py
```

Bảng 10: Các lỗi thường gặp khi chạy bài thực hành này, nguyên nhân và cách xử lý.

Thông báo lỗi	Nguyên nhân và cách xử lý
FileNotFoundError, không tìm thấy data/gia_nha.csv	Lỗi này xảy ra khi chạy Python từ bên trong thư mục code, vì lúc đó đường dẫn data/gia_nha.csv trở sang một chỗ không có tệp. Hãy quay về thư mục gốc của bài rồi chạy lại bằng <code>python code/b1_doc_du_lieu.py</code> .
ModuleNotFoundError: No module named 'sklearn'	Máy chưa cài thư viện, hoặc môi trường ảo chưa được bật trước khi chạy. Lưu ý tên gói khi cài là <code>scikit-learn</code> nhưng tên khi <code>import</code> lại là <code>sklearn</code> , hai tên này khác nhau nên rất dễ nhầm.
ModuleNotFoundError: No module named 'pandas'	Cùng nguyên nhân với dòng ngay phía trên. Hãy bật môi trường ảo rồi cài lại đủ bốn thư viện của bài.
'python' is not recognized as an internal or external command	Windows không biết tìm Python ở đâu, vì lúc cài chưa đánh dấu mục thêm Python vào PATH. Cách nhanh nhất là chạy lại bộ cài Python và nhớ đánh dấu mục đó.
'pip' is not recognized	Windows không tìm thấy <code>pip</code> , thường vì lúc cài Python sinh viên chưa đánh dấu mục thêm Python vào PATH. Cách chắc chắn nhất là gõ <code>python -m pip install ...</code> thay cho <code>pip install ...</code> , vì cách này gọi <code>pip</code> qua chính Python đang chạy. Nếu <code>python</code> cũng không chạy được thì phải cài lại Python và nhớ đánh dấu ô đó.
ValueError: Expected a 2-dimensional container but got pandas.Series	Cách viết <code>df["dien_tich"]</code> với một cặp ngoặc vuông trả về một dãy số một chiều. Thư viện <code>scikit-learn</code> cần bảng hai chiều, nên phải viết <code>df[["dien_tich"]]</code> với hai cặp ngoặc vuông.
KeyError: 'dien_tich'	Tên cột bị gõ sai, ví dụ thiếu dấu gạch dưới hoặc viết hoa chữ đầu. Hãy in <code>df.columns</code> ra xem tên cột thật rồi chép lại cho đúng từng ký tự.
SyntaxError: invalid syntax	Câu lệnh sai cấu trúc, hay gặp nhất là thiếu dấu hai chấm cuối dòng <code>for</code> hoặc <code>if</code> , hoặc thiếu một dấu ngoặc đóng. Hãy xem dòng mà Python chỉ ra, và xem cả dòng ngay phía trên nó, vì một ngoặc thiếu ở dòng trên làm Python báo lỗi ở dòng dưới.

Xem tiếp ở trang sau

Thông báo lỗi	Nguyên nhân và cách xử lý
<code>IndentationError</code>	Các dòng trong cùng một khối lùi đầu dòng không bằng nhau, thường do trộn lẫn phím <code>Tab</code> với dấu cách. Hãy đặt trình soạn thảo luôn dùng 4 dấu cách và gỡ lại phần bị lỗi.
<code>NameError: name 'np' is not defined</code>	Tên <code>np</code> được dùng trong mã nhưng dòng <code>import numpy as np</code> ở đầu tệp bị thiếu. Lỗi này cũng xảy ra với <code>pd</code> và <code>plt</code> vì cùng một lý do.
<code>AttributeError</code> , đối tượng <code>LinearRegression</code> chưa có <code>coef_</code>	Hệ số được đọc ra khi mô hình còn chưa học gì cả. Phải gọi <code>model.fit(X, y)</code> trước, sau đó mới đọc được <code>model.coef_</code> và <code>model.intercept_</code> .
<code>NotFittedError</code> , mô hình chưa được huấn luyện	Lệnh <code>.predict</code> được gọi trước khi gọi <code>.fit</code> . Thứ tự luôn là tạo mô hình, rồi <code>.fit</code> , rồi mới tới <code>.predict</code> .
<code>UnicodeEncodeError</code> , không mã hóa được ký tự	Cửa sổ lệnh của Windows đang dùng bảng mã cũ nên không in được chữ tiếng Việt có dấu. Hãy gõ <code>chcp 65001</code> để chuyển sang bảng mã UTF-8 rồi chạy lại, hoặc viết các câu <code>print</code> bằng chữ không dấu.
<code>FileNotFoundError</code> khi gọi <code>plt.savefig</code>	Thư mục để lưu hình chưa tồn tại, vì <code>savefig</code> chỉ tạo tệp chứ không tạo thư mục. Hãy tạo sẵn thư mục <code>figures</code> rồi chạy lại.
Chương trình chạy xong nhưng không thấy đồ thị	Khi chạy bằng <code>python</code> từ cửa sổ lệnh, hình chỉ hiện ra nếu có <code>plt.show()</code> , còn muốn lưu ra tệp thì dùng <code>plt.savefig</code> . Gọi <code>plt.savefig</code> sau <code>plt.show()</code> sẽ cho ra tệp trắng, nên hãy lưu hình trước rồi mới hiện lên.
Kết quả R^2 khác con số trong tài liệu	Tham số <code>random_state=42</code> trong <code>train_test_split</code> bị bỏ quên, nên mỗi lần chạy máy lại bốc ra 12 căn kiểm tra khác nhau. Thêm tham số đó vào thì R^2 sẽ đúng bằng 0.9622 như trong bảng 9.
Cửa sổ lệnh hiện ra rồi tắt ngay	Tệp <code>.py</code> được chạy bằng cách nháy đúp chuột thay vì chạy từ cửa sổ lệnh. Hãy mở cửa sổ lệnh trước, rồi gõ <code>python</code> kèm tên tệp, như vậy kết quả mới ở lại trên màn hình để đọc.

Đọc thông báo lỗi từ dòng cuối lên

Khi chương trình dừng vì lỗi, Python in ra một khối chữ dài gọi là dấu vết gọi hàm (*traceback*). Hãy đọc dòng cuối cùng trước tiên, vì chỉ dòng đó mới nói rõ loại lỗi và mô tả ngắn gọn chuyện gì đã sai. Các dòng phía trên chỉ kể lại đường đi của chương trình cho tới lúc gặp lỗi, và phần lớn nằm trong mã của thư viện chứ không phải mã của sinh viên. Sau khi hiểu dòng cuối, sinh viên dò ngược lên tìm dòng nào có tên tệp của mình kèm số dòng. Đó chính là chỗ cần sửa. Thói quen nhỏ này tiết kiệm cho sinh viên rất nhiều thời gian trong suốt môn học.